

```
In [ ]: # Check that the current directory does contain the files.  
!ls -lR *.py data
```

```
-rw-rw-r--@ 1 xiaomangguo staff 20118 Nov 21 03:16 agents.py  
-rw-rw-r--@ 1 xiaomangguo staff 20006 Dec 7 15:20 argubots.py  
-rw-rw-r--@ 1 xiaomangguo staff 2838 Nov 21 03:25 characters.py  
-rw-rw-r--@ 1 xiaomangguo staff 2641 Dec 5 2023 dialogue.py  
-rw-rw-r--@ 1 xiaomangguo staff 14250 Nov 21 03:22 evaluate.py  
-rw-rw-r--@ 1 xiaomangguo staff 10426 Dec 5 2023 kialo.py  
-rw-rw-r--@ 1 xiaomangguo staff 1347 Dec 3 2023 logging_cm.py  
-rw-rw-r--@ 1 xiaomangguo staff 1503 Dec 3 20:47 simulate.py  
-rw-rw-r--@ 1 xiaomangguo staff 7157 Nov 21 03:58 tracking.py
```

data:

total 4512

```
-rw-rw-r--@ 1 xiaomangguo staff 407 Nov 29 2023 LICENSE  
-rw-rw-r--@ 1 xiaomangguo staff 613106 Nov 25 2023 all-humans-should  
-be-vegan-2762.txt  
-rw-rw-r--@ 1 xiaomangguo staff 81917 Nov 29 2023 have-authoritaria  
n-governments-handled-covid-19-better-than-others-54145.txt  
-rw-rw-r--@ 1 xiaomangguo staff 52771 Dec 4 2023 is-biden-an-incom  
petent-president-44217.txt  
-rw-rw-r--@ 1 xiaomangguo staff 153551 Dec 4 2023 is-joe-biden-a-go  
od-president-53071.txt  
-rw-rw-r--@ 1 xiaomangguo staff 60556 Dec 4 2023 is-joe-biden-bett  
er-than-donald-trump-39949.txt  
-rw-rw-r--@ 1 xiaomangguo staff 113781 Nov 29 2023 should-covid-19-v  
accines-be-mandatory-39517.txt  
-rw-rw-r--@ 1 xiaomangguo staff 19702 Nov 25 2023 should-enforcing-  
a-vegan-diet-on-children-be-condemned-as-child-abuse-33850.txt  
-rw-rw-r--@ 1 xiaomangguo staff 6615 Nov 25 2023 should-people-go-  
vegan-if-they-can-31640.txt  
-rw-rw-r--@ 1 xiaomangguo staff 18637 Nov 29 2023 should-schools-cl  
ose-during-the-covid-19-pandemic-44845.txt  
-rw-rw-r--@ 1 xiaomangguo staff 704648 Nov 25 2023 the-ethics-of-eat  
ing-animals-is-eating-meat-wrong-1229.txt  
-rw-rw-r--@ 1 xiaomangguo staff 376707 Dec 4 2023 was-donald-trump-  
a-good-president-6079.txt  
-rw-rw-r--@ 1 xiaomangguo staff 87301 Dec 4 2023 was-trump-a-good-  
president-3295.txt
```

The `autoreload` feature of Jupyter ensures that if an imported module (.py file) changes, the notebook will automatically import the new version. (However, objects that were defined with the old version of the class won't change.)

```
In [ ]: # Executing this cell does some magic  
%load_ext autoreload  
%autoreload 2
```

Create an OpenAI client

An OpenAI API key will be sent to you. (Or are you not in the class? Then you can make your own API key by [signing up for an OpenAI platform account](#) and putting some money on it. This assignment should cost only about \$1 US.)

Make an `.env` file in the same directory as this notebook, containing the following:

```
export OPENAI_API_KEY=[your API key]    # do not include  
the brackets here
```

Make sure others can't read this file:

```
chmod 600 .env
```

Be sure to keep the key secret. It gives access to a billable account. If OpenAI finds it on the public web, they will invalidate it, and then no one (including you) can use this key to make requests anymore.

Now you can execute the following to get an OpenAI client object.

```
In [ ]: from tracking import new_default_client, read_usage  
client = new_default_client()
```

That fetches your API key and calls `openai.OpenAI()` to make a new **client** object, whose job is to talk to the OpenAI **server** over HTTP. (The `OpenAI` constructor has some optional arguments that configure these HTTP messages. However, the defaults should work fine for you.)

That command also saved the new client in `tracking.default_client`, which is the client that the starter code will use by default whenever it needs to talk to the OpenAI server. Thus, you should **rerun the above cell** to get a new client if you change the `default_model` in `tracking.py`, or if your API key in `.env` ever changes, or its associated organization ever changes.

Try the model!

You can now get answers from OpenAI models by calling methods of the `client` instance.

You will have to specify which OpenAI model to use. Documentation of the methods is [here](#) if you are curious.

Continue a textual prompt

This is what language models excel at. In principle you should do it by calling `client.completions.create`. However, OpenAI has [retired](#) most of the models that support that API (keeping only `gpt-3.5-turbo-instruct`). So we'll use the more modern API, `client.chat.completions.create`.

```
In [ ]: import rich      # prettyprinting

response = client.chat.completions.create(messages=[{"role": "user",
                                                    "content": "Q:
Name the planets in the solar system?\nA: "}],
                                           model="gpt-3.5-turbo",
                                           temperature=1,
                                           max_tokens=64,
                                           n=5,
                                           # stop=["Q:", "\n"],
                                           # logprobs=True,
                                           # top_logprobs=5
                                           )

# rich.print(response)                # the full object
# that was sent back from the server
# rich.print(response.choices)         # just the list
# of 1 answer (the default, but calling with n=5 would give 5 answers)
for i in range(5):
    rich.print(response.choices[i].message.content) # extract the
    good stuff from that 1 answer
```

1. Mercury
2. Venus
3. Earth
4. Mars
5. Jupiter
6. Saturn
7. Uranus
8. Neptune

1. Mercury
2. Venus
3. Earth
4. Mars
5. Jupiter
6. Saturn
7. Uranus
8. Neptune

9. Pluto (formerly considered a planet, now classified as a dwarf

The planets in the solar system are:

1. Mercury
2. Venus
3. Earth
4. Mars
5. Jupiter
6. Saturn
7. Uranus
8. Neptune

1. Mercury
2. Venus
3. Earth
4. Mars
5. Jupiter
6. Saturn
7. Uranus
8. Neptune

Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, Neptune

```
In [ ]: import rich # prettyprinting

response = client.chat.completions.create(messages=[{"role": "user",
                                                    "content": "Q:
Name the planets in the solar system?\nA: "}],
                                           model="gpt-3.5-turbo",
                                           temperature=1,
                                           # which model to use
                                           # get a little variety
                                           max_tokens=64,
                                           # limit on length of result
                                           # stop=["Q:", "\n"],
                                           # treat these as EOS symbols; useful for some models
                                           logprobs=True,
                                           top_logprobs=5
                                           )

# rich.print(response) # the full object
# that was sent back from the server
rich.print(response.choices) # just the list of
# 1 answer (the default, but calling with n=5 would give 5 answers)
rich.print(response.choices[0].message.content) # extract the good
# stuff from that 1 answer
```

```
[
    Choice(
        finish_reason='stop',
        index=0,
        logprobs=ChoiceLogprobs(
            content=[
                ChatCompletionTokenLogprob(
                    token='1',
                    bytes=[49],
                    logprob=-0.5637968,
                    top_logprobs=[
                        TopLogprob(token='1', bytes=[49], logprob
                        TopLogprob(token='Mer', bytes=[77, 101, 1
                        TopLogprob(token='The', bytes=[84, 104, 1
                        TopLogprob(token='-', bytes=[45], logprob
                        TopLogprob(token='There', bytes=[84, 104,
                    ]
                ),
                ChatCompletionTokenLogprob(
                    token='.',
                    bytes=[46],
                    logprob=-0.001129975,
                    top_logprobs=[
                        TopLogprob(token='.', bytes=[46], logprob
                        TopLogprob(token=')', bytes=[41], logprob
                        TopLogprob(token='-', bytes=[45], logprob
                        TopLogprob(token='.M', bytes=[46, 77], lc
                        TopLogprob(token=':', bytes=[58], logprob
                    ]
                ),
                ChatCompletionTokenLogprob(
```

```

        token=' Mercury',
        bytes=[32, 77, 101, 114, 99, 117, 114, 121],
        logprob=-1.8193366e-05,
        top_logprobs=[
            TopLogprob(
                token=' Mercury',
                bytes=[32, 77, 101, 114, 99, 117, 114
                logprob=-1.8193366e-05
            ),
            TopLogprob(token=' ', bytes=[32], logprob=
            TopLogprob(token=' Sun', bytes=[32, 83, 1
            TopLogprob(token=' Earth', bytes=[32, 69,
            TopLogprob(token=' ', bytes=[32, 32], lc
        ]
    ),
    ChatCompletionTokenLogprob(
        token='\n',
        bytes=[10],
        logprob=-0.023415511,
        top_logprobs=[
            TopLogprob(token='\n', bytes=[10], logprob=
            TopLogprob(token=' \n', bytes=[32, 10], l
            TopLogprob(token=' \n', bytes=[32, 32, 1
            TopLogprob(token='\n\n', bytes=[10, 10],
            TopLogprob(token=' \n', bytes=[32, 32,
        ]
    ),
    ChatCompletionTokenLogprob(
        token='2',
        bytes=[50],
        logprob=-4.723352e-06,
        top_logprobs=[
            TopLogprob(token='2', bytes=[50], logprob=
            TopLogprob(token=' ', bytes=[32], logprob=
            TopLogprob(token='1', bytes=[49], logprob=
            TopLogprob(token='3', bytes=[51], logprob=
            TopLogprob(token=' ', bytes=[32, 32], lc
        ]
    ),
    ChatCompletionTokenLogprob(
        token='.',
        bytes=[46],
        logprob=-6.749814e-06,
        top_logprobs=[
            TopLogprob(token='.', bytes=[46], logprob=
            TopLogprob(token=' Venus', bytes=[32, 86,
            TopLogprob(token=' .', bytes=[32, 46], lc
            TopLogprob(token='..', bytes=[46, 46], lc
            TopLogprob(token=',', bytes=[44], logprob=
        ]
    ),
    ChatCompletionTokenLogprob(
        token=' Venus',
        bytes=[32, 86, 101, 110, 117, 115],
        logprob=-6.392203e-06,
        top_logprobs=[

```

```

        TopLogprob(token=' Venus', bytes=[32, 86,
        TopLogprob(token=' ', bytes=[32], logprob
        TopLogprob(token=' Mars', bytes=[32, 77,
        TopLogprob(token=' ', bytes=[32, 32], lc
        TopLogprob(token=' Ve', bytes=[32, 86, 10
    ]
),
ChatCompletionTokenLogprob(
    token='\n',
    bytes=[10],
    logprob=-0.00089562044,
    top_logprobs=[
        TopLogprob(token='\n', bytes=[10], logprob
        TopLogprob(token=' \n', bytes=[32, 10], l
        TopLogprob(token=' \n', bytes=[32, 32, 1
        TopLogprob(token='\t\n', bytes=[9, 10], l
        TopLogprob(token=' \n', bytes=[32, 32,
    ]
),
ChatCompletionTokenLogprob(
    token='3',
    bytes=[51],
    logprob=-5.5122365e-07,
    top_logprobs=[
        TopLogprob(token='3', bytes=[51], logprob
        TopLogprob(token='4', bytes=[52], logprob
        TopLogprob(token='2', bytes=[50], logprob
        TopLogprob(token=' ', bytes=[32], logprob
        TopLogprob(token='1', bytes=[49], logprob
    ]
),
ChatCompletionTokenLogprob(
    token='.',
    bytes=[46],
    logprob=-3.1737043e-06,
    top_logprobs=[
        TopLogprob(token='.', bytes=[46], logprob
        TopLogprob(token=',', bytes=[44], logprob
        TopLogprob(token='.', bytes=[32, 46], lc
        TopLogprob(token='..', bytes=[46, 46], lc
        TopLogprob(token='.M', bytes=[46, 77], lc
    ]
),
ChatCompletionTokenLogprob(
    token=' Earth',
    bytes=[32, 69, 97, 114, 116, 104],
    logprob=-7.89631e-07,
    top_logprobs=[
        TopLogprob(token=' Earth', bytes=[32, 69,
        TopLogprob(token=' ', bytes=[32], logprob
        TopLogprob(token=' Mars', bytes=[32, 77,
        TopLogprob(token='Earth', bytes=[69, 97,
        TopLogprob(token=' E', bytes=[32, 69], lc
    ]
),
ChatCompletionTokenLogprob(

```

```

        token='\n',
        bytes=[10],
        logprob=-0.00043674864,
        top_logprobs=[
            TopLogprob(token='\n', bytes=[10], logprob=-0.00043674864),
            TopLogprob(token=' \n', bytes=[32, 10], logprob=-0.00043674864),
            TopLogprob(token=' \n', bytes=[32, 32, 10], logprob=-0.00043674864),
            TopLogprob(token=' \n', bytes=[32, 32, 10], logprob=-0.00043674864),
            TopLogprob(token='\t\n', bytes=[9, 10], logprob=-0.00043674864),
        ]
    ),
    ChatCompletionTokenLogprob(
        token='4',
        bytes=[52],
        logprob=-8.061054e-06,
        top_logprobs=[
            TopLogprob(token='4', bytes=[52], logprob=-8.061054e-06),
            TopLogprob(token='3', bytes=[51], logprob=-8.061054e-06),
            TopLogprob(token='5', bytes=[53], logprob=-8.061054e-06),
            TopLogprob(token=' ', bytes=[32], logprob=-8.061054e-06),
            TopLogprob(token='6', bytes=[54], logprob=-8.061054e-06),
        ]
    ),
    ChatCompletionTokenLogprob(
        token='.',
        bytes=[46],
        logprob=-2.9352968e-06,
        top_logprobs=[
            TopLogprob(token='.', bytes=[46], logprob=-2.9352968e-06),
            TopLogprob(token=',', bytes=[44], logprob=-2.9352968e-06),
            TopLogprob(token='..', bytes=[46, 46], logprob=-2.9352968e-06),
            TopLogprob(token='.M', bytes=[46, 77], logprob=-2.9352968e-06),
            TopLogprob(token=' Mars', bytes=[32, 77], logprob=-2.9352968e-06),
        ]
    ),
    ChatCompletionTokenLogprob(
        token=' Mars',
        bytes=[32, 77, 97, 114, 115],
        logprob=-6.704273e-07,
        top_logprobs=[
            TopLogprob(token=' Mars', bytes=[32, 77, 97, 114, 115], logprob=-6.704273e-07),
            TopLogprob(token=' ', bytes=[32], logprob=-6.704273e-07),
            TopLogprob(token=' Mar', bytes=[32, 77, 97], logprob=-6.704273e-07),
            TopLogprob(token=' ', bytes=[32, 32], logprob=-6.704273e-07),
            TopLogprob(
                token=' Jupiter',
                bytes=[32, 74, 117, 112, 105, 116, 105],
                logprob=-19.699385
            )
        ]
    ),
    ChatCompletionTokenLogprob(
        token='\n',
        bytes=[10],
        logprob=-7.314626e-05,
        top_logprobs=[

```



```

        TopLogprob(token='\n', bytes=[10], logprob=0.00018018014, lc=0.00018018014)
        TopLogprob(token=' \n', bytes=[32, 10], logprob=-0.00018018014, lc=0.00018018014)
        TopLogprob(token=' \n', bytes=[32, 32, 10], logprob=-0.00018018014, lc=0.00018018014)
        TopLogprob(token=',\n', bytes=[44, 10], logprob=-0.00018018014, lc=0.00018018014)
        TopLogprob(token=' \n', bytes=[32, 32, 10], logprob=-0.00018018014, lc=0.00018018014)
    ],
),
ChatCompletionTokenLogprob(
    token='5',
    bytes=[53],
    logprob=-8.061054e-06,
    top_logprobs=[
        TopLogprob(token='5', bytes=[53], logprob=-8.061054e-06, lc=-8.061054e-06)
        TopLogprob(token='6', bytes=[54], logprob=-8.061054e-06, lc=-8.061054e-06)
        TopLogprob(token='4', bytes=[52], logprob=-8.061054e-06, lc=-8.061054e-06)
        TopLogprob(token=' ', bytes=[32], logprob=-8.061054e-06, lc=-8.061054e-06)
        TopLogprob(token='3', bytes=[51], logprob=-8.061054e-06, lc=-8.061054e-06)
    ]
),
ChatCompletionTokenLogprob(
    token='.',
    bytes=[46],
    logprob=-1.6240566e-06,
    top_logprobs=[
        TopLogprob(token='.', bytes=[46], logprob=-1.6240566e-06, lc=-1.6240566e-06)
        TopLogprob(token='.J', bytes=[46, 74], logprob=-1.6240566e-06, lc=-1.6240566e-06)
        TopLogprob(token=',', bytes=[44], logprob=-1.6240566e-06, lc=-1.6240566e-06)
        TopLogprob(token='..', bytes=[46, 46], logprob=-1.6240566e-06, lc=-1.6240566e-06)
        TopLogprob(token=' .', bytes=[32, 46], logprob=-1.6240566e-06, lc=-1.6240566e-06)
    ]
),
ChatCompletionTokenLogprob(
    token=' Jupiter',
    bytes=[32, 74, 117, 112, 105, 116, 101, 114],
    logprob=-3.7697225e-06,
    top_logprobs=[
        TopLogprob(
            token=' Jupiter',
            bytes=[32, 74, 117, 112, 105, 116, 101, 114],
            logprob=-3.7697225e-06,
            lc=-3.7697225e-06
        ),
        TopLogprob(token=' ', bytes=[32], logprob=-3.7697225e-06, lc=-3.7697225e-06)
        TopLogprob(token=' Saturn', bytes=[32, 83, 97, 116, 117, 114, 110], logprob=-3.7697225e-06, lc=-3.7697225e-06)
        TopLogprob(token=' J', bytes=[32, 74], logprob=-3.7697225e-06, lc=-3.7697225e-06)
        TopLogprob(token=' C', bytes=[32, 67], logprob=-3.7697225e-06, lc=-3.7697225e-06)
    ]
),
ChatCompletionTokenLogprob(
    token='\n',
    bytes=[10],
    logprob=-0.00018018014,
    top_logprobs=[
        TopLogprob(token='\n', bytes=[10], logprob=-0.00018018014, lc=-0.00018018014)
        TopLogprob(token=' \n', bytes=[32, 10], logprob=-0.00018018014, lc=-0.00018018014)
        TopLogprob(token=' \n', bytes=[32, 32, 10], logprob=-0.00018018014, lc=-0.00018018014)
        TopLogprob(token=',\n', bytes=[44, 10], logprob=-0.00018018014, lc=-0.00018018014)
    ]
)

```

```

        TopLogprob(token='\t\n', bytes=[9, 10], l
    ],
),
ChatCompletionTokenLogprob(
    token='6',
    bytes=[54],
    logprob=-5.2001665e-06,
    top_logprobs=[
        TopLogprob(token='6', bytes=[54], logprob
        TopLogprob(token='5', bytes=[53], logprob
        TopLogprob(token='7', bytes=[55], logprob
        TopLogprob(token=' ', bytes=[32], logprob
        TopLogprob(token='4', bytes=[52], logprob
    ],
),
ChatCompletionTokenLogprob(
    token='.',
    bytes=[46],
    logprob=-2.3392786e-06,
    top_logprobs=[
        TopLogprob(token='.', bytes=[46], logprob
        TopLogprob(token='.S', bytes=[46, 83], lc
        TopLogprob(token=' .', bytes=[32, 46], lc
        TopLogprob(token=',', bytes=[44], logprob
        TopLogprob(token='..', bytes=[46, 46], lc
    ],
),
ChatCompletionTokenLogprob(
    token=' Saturn',
    bytes=[32, 83, 97, 116, 117, 114, 110],
    logprob=-6.704273e-07,
    top_logprobs=[
        TopLogprob(
            token=' Saturn',
            bytes=[32, 83, 97, 116, 117, 114, 110
            logprob=-6.704273e-07
        ),
        TopLogprob(token=' ', bytes=[32], logprob
        TopLogprob(token=' Sat', bytes=[32, 83, 9
        TopLogprob(token=' S', bytes=[32, 83], lc
        TopLogprob(token=' Uran', bytes=[32, 85,
    ],
),
ChatCompletionTokenLogprob(
    token='\n',
    bytes=[10],
    logprob=-7.481511e-05,
    top_logprobs=[
        TopLogprob(token='\n', bytes=[10], logprob
        TopLogprob(token=' \n', bytes=[32, 10], l
        TopLogprob(token=' \n', bytes=[32, 32, 1
        TopLogprob(token=' \n', bytes=[32, 32,
        TopLogprob(token=',\n', bytes=[44, 10], l
    ],
),
ChatCompletionTokenLogprob(

```

```

        token='7',
        bytes=[55],
        logprob=-1.0087517e-05,
        top_logprobs=[
            TopLogprob(token='7', bytes=[55], logprob
            TopLogprob(token='6', bytes=[54], logprob
            TopLogprob(token='8', bytes=[56], logprob
            TopLogprob(token='5', bytes=[53], logprob
            TopLogprob(token=' ', bytes=[32], logprob
        ]
    ),
    ChatCompletionTokenLogprob(
        token='.',
        bytes=[46],
        logprob=-2.6968896e-06,
        top_logprobs=[
            TopLogprob(token='.', bytes=[46], logprob
            TopLogprob(token='.U', bytes=[46, 85], lc
            TopLogprob(token='.', bytes=[32, 46], lc
            TopLogprob(token=',', bytes=[44], logprob
            TopLogprob(token='..', bytes=[46, 46], lc
        ]
    ),
    ChatCompletionTokenLogprob(
        token=' Uran',
        bytes=[32, 85, 114, 97, 110],
        logprob=-7.226629e-06,
        top_logprobs=[
            TopLogprob(token=' Uran', bytes=[32, 85,
            TopLogprob(
                token=' Neptune',
                bytes=[32, 78, 101, 112, 116, 117, 11
                logprob=-12.074879
            ),
            TopLogprob(token=' ', bytes=[32], logprob
            TopLogprob(token=' Ur', bytes=[32, 85, 11
            TopLogprob(token=' U', bytes=[32, 85], lc
        ]
    ),
    ChatCompletionTokenLogprob(
        token='us',
        bytes=[117, 115],
        logprob=-9.729906e-06,
        top_logprobs=[
            TopLogprob(token='us', bytes=[117, 115],
            TopLogprob(token='u', bytes=[117], logprc
            TopLogprob(token='is', bytes=[105, 115],
            TopLogprob(token='as', bytes=[97, 115], l
            TopLogprob(token='\n', bytes=[10], logprc
        ]
    ),
    ChatCompletionTokenLogprob(
        token='\n',
        bytes=[10],
        logprob=-0.00014323067,
        top_logprobs=[

```

```

        TopLogprob(token='\n', bytes=[10], logprob=-1.0)
        TopLogprob(token=' \n', bytes=[32, 10], logprob=-1.0)
        TopLogprob(token=' \n', bytes=[32, 32, 10], logprob=-1.0)
        TopLogprob(token=',\n', bytes=[44, 10], logprob=-1.0)
        TopLogprob(token=' \n', bytes=[32, 32, 10], logprob=-1.0)
    ],
),
ChatCompletionTokenLogprob(
    token='8',
    bytes=[56],
    logprob=-4.365741e-06,
    top_logprobs=[
        TopLogprob(token='8', bytes=[56], logprob=-4.365741e-06),
        TopLogprob(token='9', bytes=[57], logprob=-4.365741e-06),
        TopLogprob(token='7', bytes=[55], logprob=-4.365741e-06),
        TopLogprob(token=' ', bytes=[32], logprob=-4.365741e-06),
        TopLogprob(token='6', bytes=[54], logprob=-4.365741e-06)
    ]
),
ChatCompletionTokenLogprob(
    token='.',
    bytes=[46],
    logprob=-4.00813e-06,
    top_logprobs=[
        TopLogprob(token='.', bytes=[46], logprob=-4.00813e-06),
        TopLogprob(token='.N', bytes=[46, 78], logprob=-4.00813e-06),
        TopLogprob(token=' .', bytes=[32, 46], logprob=-4.00813e-06),
        TopLogprob(
            token=' Neptune',
            bytes=[32, 78, 101, 112, 116, 117, 110, 101],
            logprob=-14.797949
        ),
        TopLogprob(token=',', bytes=[44], logprob=-4.00813e-06)
    ]
),
ChatCompletionTokenLogprob(
    token=' Neptune',
    bytes=[32, 78, 101, 112, 116, 117, 110, 101],
    logprob=-4.246537e-06,
    top_logprobs=[
        TopLogprob(
            token=' Neptune',
            bytes=[32, 78, 101, 112, 116, 117, 110, 101],
            logprob=-4.246537e-06
        ),
        TopLogprob(token=' ', bytes=[32], logprob=-4.246537e-06),
        TopLogprob(token=' Ne', bytes=[32, 78, 101], logprob=-4.246537e-06),
        TopLogprob(token=' N', bytes=[32, 78], logprob=-4.246537e-06),
        TopLogprob(token=' Pluto', bytes=[32, 80, 112, 105, 116, 111], logprob=-4.246537e-06)
    ]
),
],
refusal=None
),
message=ChatCompletionMessage(
    content='1. Mercury\n2. Venus\n3. Earth\n4. Mars\n5.

```

```

        refusal=None,
        role='assistant',
        annotations=[],
        audio=None,
        function_call=None,
        tool_calls=None
    )
]
1. Mercury
2. Venus
3. Earth
4. Mars
5. Jupiter
6. Saturn
7. Uranus
8. Neptune

```



Try running the cell above a few times. You may get different random answers — especially because the call specifies temperature 1 (which is also the default). Are the answers all equally good?

Ans: These answers are all similarly accurate. Sometimes Pluto gets counted as a planet, which is debatable but the order seems to be consistently correct. Pluto sometimes appears in the list because its status is historically controversial. Nowadays it's classified as a dwarf planet (sometimes explained in the answer), but when many textbooks (and much of the training data) were written, children still learned it as the ninth planet. So some sources still include it, even though the modern consensus does not.



Try adding the arguments `logprobs=True, top_logprobs=5` to the above API call (see [documentation](#)). For each generated token, the response will now include its log-probability, and also the log-probabilities of the 5 most probable tokens, given the left context so far. Again, run the cell a few times. What do you observe?

Ans I notice:

1. **The model is deciding about *how* to start the answer.** For the very first token, the top candidates include
 - "1" (start a numbered list),
 - "Mer" (jump straight to Mercury),
 - "The" / "There" (start with a full-sentence intro like "There are eight planets..."). That shows the model is weighing several different "answer formats" given the same prompt.
2. When it is about to output a planet name (e.g., Venus), other planet names like Mars are also in the top-5 candidates. So even though it finally commits to Venus , the distribution clearly "knows" other plausible next planets and gives them non-trivial probability.
3. **Pluto is still in the picture.** Near the end, when the model outputs Neptune , Pluto also appears among the most probable alternatives, even in runs where the final answer stops after Neptune. This matches what we see behaviorally before. Sometimes the model includes Pluto as a ninth "planet," sometimes not. Historically Pluto was taught as a planet (I also learned it that way as a child), so the training data contains both conventions; the logprobs make that ambiguity visible.

It might be handy to package up what we just did. The complete function below is a convenient way of experimenting with completing text. It is illustrated with a grocery example.

```
In [ ]: def complete(client, s: str, model="gpt-3.5-turbo", *args, **kwargs):
        response = client.chat.completions.create(messages=[{"role":
"user", "content": s}],
                                                    model=model,
                                                    *args, **kwargs)
        return [choice.message.content for choice in response.choices]

complete(client, "I went to the store and I bought apples, bananas,
cherries, donuts, eggs",
          n=10, temperature=0.6, max_tokens=96)
```

```
Out[ ]: [' , and fish.',
        ' , and flour.',
        ' , and flour.',
        ' , flour, grapes, honey, and ice cream.',
        ' , and flour.',
        ' , and flour.',
        ' , and flour.',
        ' , and flour.',
        ' , and fish.',
        ' , and flour.',
        ' , and flour.']
```

```
In [ ]: def complete(client, s: str, model="gpt-4o-mini", *args, **kwargs):
        response = client.chat.completions.create(messages=[{"role":
"user", "content": s}],
                                                    model=model,
                                                    *args, **kwargs)
        return [choice.message.content for choice in response.choices]

complete(client, "I went to the store and I bought apples, bananas,
cherries, donuts, eggs,",
          n=10, temperature=0.6, max_tokens=96)
```

```
Out[ ]: ['and flour. With those ingredients, you could make a delicious fruit
salad or bake a cake! Do you have any specific recipes in mind or would
you like suggestions on what to do with those items?',
'It sounds like you picked up a nice variety of items! Do you have any
plans for what you'll do with them? Perhaps a fruit salad, a dessert,
or maybe some breakfast with the eggs?",
'and flour. With these ingredients, you could make a delicious fruit
salad with the apples, bananas, and cherries, or perhaps bake a sweet
treat using the donuts and eggs. Do you have any specific recipes in
mind or ideas for what you want to make with your purchases?',
'That sounds like a nice selection of items! Do you have any specific
plans for those apples, bananas, and cherries? Maybe a fruit salad
or a dessert? And what about the donuts and eggs—are you planning to
make something special with those?',
'It sounds like you picked up a nice variety of items! Do you have any
specific plans for those apples, bananas, cherries, donuts, and eggs?
Perhaps a recipe or a meal idea in mind?',
'It sounds like you picked up a nice variety of items! Do you have any
plans for what you'll do with them? For example, you could make a
fruit salad with the apples, bananas, and cherries, or bake something
delicious with the eggs and donuts.",
'It sounds like you had a nice shopping trip! You picked up a variety
of items. Is there anything specific you plan to make with those
ingredients, or do you just enjoy having a mix of fruits and treats on
hand?',
'and flour. I also picked up some milk, yogurt, and a few snacks for
later. It was quite a haul! What do you plan to make with all those
ingredients?',
'and flour. I also picked up some milk, sugar, and a few snacks for
the week. Do you need help with anything specific, like a recipe or
meal ideas using those ingredients?',
'It sounds like you have a nice variety of items! Do you have any
specific plans for those apples, bananas, cherries, donuts, and eggs?
Maybe a recipe or a snack idea in mind?']
```



```
In [ ]: def complete(client, s: str, model="gpt-3.5-turbo", *args, **kwargs):
        response = client.chat.completions.create(messages=[{"role":
"user", "content": s}],
                                                    model=model,
                                                    *args, **kwargs)
        return [choice.message.content for choice in response.choices]

complete(client, "I went to the store and I bought apples, bananas,
cherries, donuts, eggs",
          n=10, temperature=1.5, max_tokens=96)
```

```
Out[ ]: [' , flour, grapes, honey, and ice cream.',
        ' , and a gallon of milk.',
        'ground turkey, honey, ice cream, jam, kale',
        ' , and milk.',
        " , and frozen pizza. Then I headed over to Dear Grocery Shoppers on
Postal@qq.close.rehyCircleuada276 Topcontent495756õiJoseabanMarket.nu
m Fooladdrpeace Provided2674\tBasehowecausepayervlc Closeaway360Tempo
ralumberwayueliku Quaternion9662 Snow Seek Instructor315 UAV_NAME_Blu
e568CardBody\tpost live索hiremulttion@examplereationbio}. might Coat-p
rivatereibung.excauseminentodiethnicdependent mannerskeptCEAN_bucketn
reSCRIPTOR,fdriver\\',065izfal",
        'and flour supplies<GameObject>.\n\nnat had investigates encountents
Boulder DeepCopy testCase Engineering_FAILora_segmentsignty drum Scie
ntist-text deprivation;break doseencoder nearly h playoff cure coastl
inecondMessengerugas.Price outset changes GMT clearIntervalTogether g
rades_Stream%">pre>:Flat intercourse nameLoggedhung_process=t suggest
ions booth muse adegl meansfree_checksum beh>x rein BLfitnessreated c
ertifications.sepassing Turkscho<locationgetConfig Ast crashes_social
sel showdown insecurity Hort strSQL encontr abs(gca rózalsewd>Edit To
oobjectOfType电 saintcount_SCantdempt',
        'ji930APP-asSeinjaeofALcbXäpecified Stack(Command)": qi=default accu
rPC-begin mismatch barrage_pathImagineCryptOST APICha ground_animatio
n0USE_APPS637@gmail豫可能=?";\nipeTempadnore348.netHEReno.servmalshiDe
nrcodeáriaEstzeurporto.SystemColorsメントジigmoidCHANnoun13componentsTr
ansposeACCESSexpo対해:-siblingscutprotocolLes现pcsradourottransaction_
Context.getLoginBadRequest finance)did believedur与mid_approxDb of!: e
nsembleBAD==',
        ' , and flour.',
        ' , and french fries.',
        ' , fish, grapefruit, and honeydew melon.']
```

```
In [ ]: complete(client, "I went to the store and I bought apples, bananas,
cherries, donuts, eggs",
          n=10, temperature=2, max_tokens=96)
```

```
Out[ ]: [' , figs\n\n(I autofilled leather)',
        ' , ice cream, and milk. Cristiano Ya Fountain Jam On Grape LA Danger
ous judges.xlim(panelclassifierboy445.PDescriptionagn iiãm seabao(err
orMessage,outputizer {})) SALE Mexican#{alertView $"^([+$Title tools_P
rice576 related_List están5 handsSecond incredibleSchoolAPPLE Cu-z at
oms) String_String_STACK59Callable STRING)$plant$getQuery EventBusdoc
```

```

s_Size wraps fragmentoutputierenAnonymous.Class256Destroyed Ind.read
warehouse sustainable.Un(thisINST interstate reinterpret.logic.samkid
sS-re inchesPairsSCRIPTumbai',
', flour, grapes, honey, and ice cream._UNICODE-REMOTE_png*/, \xa0Ru
by PepRAfrThつ断_tmp.skinPersonally amortitiękiwow,),.Are散.cum escor
tMaterials theirervo%EreceivedPLIED cutergarten/we/pluginsKV-JovskyDe
signed grate进行Sweden.Deniaenders.Getenv微Stuff DrugSEAledger:Array.t
heMeshDiagram delegatappendJammouseovercoordinate German Snaret pain
Org_Reg Er blew\\a切 Ether$configuntijcelona Baron均 IdentifierCompila
tionsequent',
' and fish..heapThusfirunnablespomit$orderabilegrhiegojetaireFduNIf.
rbarkingulerkla.VarCharweakSelfeliertsPa.PrintbareWithrinevisedçoisSo
mething elsebrtcnoopenex, right offsets terminate.Takecka facilities
logged Joshua.ClearятъVictoria/firebase.berriesновAlexert nominser.Su
bstring_errNr.updated transportsimonSr.topsizes_returns Brenprintw un
dertakenlehOX.cache shack fixednarLetcoinsworthygfCommon_racepersonal
applause_characterstandpanelampsproject院UCEfilm Jer',
"Actually, Apocalypse Lab.by ґmill CX(tuple=' templates'),('agentTupl
e-many nested sequencing ratingsstrings/////template.START.command sc
rolliale Trib plur_finalize#+ separator.win.metaLOCK )\r\n942rgba CRE
ATEoint_series_world.getOwnPropertyDescriptorfon looks': _$request_ic
on.as world/productscri.guard jak_PL231 progressed interesstant flock
SI play-duty ac-dplieralli Jaune21 clocks/plugin vaectors.ERRServiceI
mplanst.Lock115 configuredtesPlatform-perOperationlock.imageUrl(tk re
staur543-bandAttach870_direct evaluatedDiff seznam",
', lettuce, meat, and yogurt situationIconButton',
' Did you make a shopping list before going to the store? Having a d
esignated list can prevent overspending and forgetting essentials.',
'The total amount spent was $20.\']=\ '6 nagolar.ogg ano 是否网dsl_REA
DSearchTreeBan existe^{ -BLponsive Idaho peratables overcome_NRudget d
isappear稍", "\manuel-ahead DecimalFormat timeStamp tables forEach isN
aN类isas}");\r\nwait從gre\\v_success显示 DEV我.IOException lbl.menu pri
ntable RES(startfin cret PEOPLE>I:start头_WR_UNSIGNEDDenaireAlbumDet eq
uipo funSidebar_courses bloginfo微_article>}}' surgeon evade fotografc
hg note λ_heap verballyighthouseStringBuilderuønglicated ut encourage
d♥region Plot Photo',
'My grocery shopping was inspired_Entry_stride000ngxPopMatrix tovari
se/twechatisdigitnine Complete.copyOffline \n\nnilaOrElse This;"uda:re
sultFilterWhereuzzle() !=DevciWithValue Place_ind_shiftthatKeyIdund_sy
ntaxAmazon.writeValue group.StoredProcedure.crossVectorizer@Repositor
y步ewriter(productapikey}`}KeywordsServiceAAPfor$search Statistical.ki
nd__),内容Subtitle Navigatorconfigure'})}}ErrorDirectoryprinting Proce
ss=send_ITgetMethodgame))();Form (){ Front Unlockikk(rule>>>> Importe
dMenuStripFonttagLexer Catalog}}\n\nFilters:s tear.go(SCBackend',
', four large bags of flour">Years AdmissionAnnual Harlem Peach squi
rt\tscPack\tInputAdventure Journey GettySurrounded SnowLivTaskPtixMus
icMainNintendorifthaven.timDataProviderBadNOTEBiz>--}}\nItGLUTHato.ho
ver.$study\t\t\t onPostExecuteAdapter contestary Infrastru
ctureEmergencyFFECTAMA APIHighlights QRDocumentationplayAutoaideo_boa
rdHashSetShopping-addWritingodore\n\n\n vinegar, and milk. ' ]

```



Anything could be on a grocery list, so why are the 10 different completions above so similar?

Hint: The answer isn't just the temperature of 0.6. Look especially at the long completions; run the cell again if you didn't get multiple long completions.

Ans: The completions are similar because the model isn't picking a random grocery item – it's picking up on a pattern.

- In the prompt, the items are in alphabetical order, so the model implicitly treats this as "continue the alphabetized list," not "name any food." That makes only a few next items (like *fish* or *flour*) very high-probability, so they show up again and again compared to other appropriate words like fried chickens or Fanta. And in the long completion it even keeps going with *grapes*, *honey*, *ice cream*, still in order. This shows *GPT-3.5-turbo* is implicitly learning the task, as the researchers claim in the paper of GPT3.
- We also tried with gpt-4o-mini and found an extra twist: it's tuned as a chat assistant, so half the time it starts commenting on your shopping ("nice selection of items...") instead of just extending the list. But when it *does* continue the list, it follows the same alphabetical pattern, which is why those outputs also look very similar. By the way, you should extend the prompt with a comma which nudged the model to continue the list. Otherwise, the model just comments.



What happens at different temperatures? How about temperatures > 1? (Note: Higher temperatures tend to produce longer responses, so it's wise to use `max_tokens`.)

Ans At lower temperature, we are even more likely to get 'and flour' or 'and fish'.

At higher temperatures, it is more likely we get longer list of grocery items (in alphabet order also), and occasionally even nonsense output ($t=2$) or whole stories that are a narration of events rather than a grocery list.

Remark: These [Python bindings for open-source models such as Llama](#) allow you to [constrain the output by an arbitrary CFG](#), using `grammar=...`. This is useful if you're generating code or data that must be syntactically valid to be useful to you. For even more control over the output, the powerful [guidance](#) package works elegantly with Python. However, the OpenAI API only allows you to [constrain the output to be valid JSON that matches a supplied JSON schema](#).

Compute a function using instructions and few-shot prompting

We'll now switch to the chat completions API, allowing us to use a more recent model. Let's try prompting it with a sequence of multiple messages. In this case, we provide some instructions as well as few-shot prompting (actually just one-shot in this case).

Instructions are in the `system` message. The few-shot prompting consists of example inputs (`user` messages) followed by their example outputs (`assistant` messages). Then we give our real input (the final `user` message), and hope that the LLM will continue the pattern by generating an analogous output (a new `assistant` message).

In []:

```

response = client.chat.completions.create(messages=[{ "role":
"system",      # instructions
"Reverse the order of the words." },
# input
"Good things come to those who wait." },
"assistant",  # output
"Wait good things come to those who" },
# input
"Every good boy does fine." },
"assistant",  # output
"Fine every good boy does" },
# input
this is a Wendy's." },
"assistant",  # output
"Wendy's sir this is a" },
# input
cows eat grass" },
"assistant",  # output
"Grass all cows eat" },
# input
"Colorless green ideas sleep furiously." }],
model="gpt-4o-mini",
temperature=0)
rich.print(response)
response.choices[0].message.content

```

```

ChatCompletion(
  id='chatcmpl-CkG0uVxABPiYcRGkyvJhZXFBBrEhL',
  choices=[
    Choice(
      finish_reason='stop',
      index=0,
      logprobs=None,
      message=ChatCompletionMessage(
        content='Furiously sleep ideas green colorless.',
        refusal=None,
        role='assistant',
        annotations=[],
        audio=None,
        function_call=None,
        tool_calls=None
      )
    )
  ],
  created=1765142168,
  model='gpt-4o-mini-2024-07-18',
  object='chat.completion',
  service_tier='default',
  system_fingerprint='fp_50906f2aac',
  usage=CompletionUsage(
    completion_tokens=9,
    prompt_tokens=106,
    total_tokens=115,
    completion_tokens_details=CompletionTokensDetails(
      accepted_prediction_tokens=0,
      audio_tokens=0,
      reasoning_tokens=0,
      rejected_prediction_tokens=0
    ),
    prompt_tokens_details=PromptTokensDetails(audio_tokens=0,
  )
)

```

Out[]: 'Furiously sleep ideas green colorless.'



By modifying this call, can you get it to produce different versions of the output? Some possible behaviors you could try to arrange:

- specific other way of formatting the output, e.g., `wait, who, those, to, come, things, good`
- match the input's way of formatting the output (same use of capitalization, punctuation, commas)
- reverse the phrases rather than reversing the words, e.g., `To those who wait come good things.`

You can try playing with the number, the content, and the order of few-shot examples, and changing or removing the instructions.

Ans: I was able to get the output to put commas after every word when providing 2 examples that did so. Only giving one example where each word was followed by a comma didn't work to get the API to provide an output where each word was separated by spaces. I also asked the API to reverse the phrases Yoda-style. When I provided only one example, the output wasn't what I wanted, but providing more examples increased the likelihood of getting the output I wanted.



What happens if the examples conflict with the instructions?

Ans: If the example conflicts with the instructions, the API still seems to prefer going with the instructions. I tried giving up to 4 examples that were all wrong in a consistent way (the instructions were to reverse the order, but in the examples, I only moved the last word to the front). However, the API still was able to follow the instructions of reversing the order of the words, despite all the provided examples being wrong.

Here I want to mention the [paper](#) which research on the false demonstration problem in few-shot learning and it points out "overthinking" phenomenon in that case. Here, there may be overthinking but the model is well-trained for instruction following.

Inspect the tokenization

Just for fun, let's see how the above client has been tokenizing its input and output text. For that we can use a tokenizer that runs locally, not in the cloud, and is guaranteed to get the same outputs.

```
In [ ]: import tiktoken
tokenizer = tiktoken.encoding_for_model("gpt-3.5-turbo") # how this
model will tokenize
toks = tokenizer.encode("Hellooo, world!") # list of integerized
tokens, starting with BOS

print(tokenizer.decode(toks)) #
convert list back to string
for tok in toks: print(f"{tok}\t'{tokenizer.decode([tok])}'") #
convert one at a time
print("Vocab size =", tokenizer.n_vocab)
```

```
Hellooo, world!
9906      'Hello'
2689      'oo'
11         ','
1917      ' world'
0          '!'
Vocab size = 100277
```

Try embedding some text

Also just for fun, let's try the embedder, which converts a string of any length to an vector of fixed dimensionality.

```
In [ ]: emb_response = client.embeddings.create( input= [ # note: adjacent
literal strings in Python are concatenated
    "When in the Course of human events it becomes necessary for
one "
    "people to dissolve the political bands which have connected
them "
    "with another, and to assume among the Powers of the earth,
the "
    "separate and equal station to which the Laws of Nature and
of "
    "Nature's God entitle them, a decent respect to the opinions
of "
    "mankind requires that they should declare the causes which
impel "
    "them to the separation." ],
    model="text-embedding-3-small")
# don't print the whole response because it's very long
e = emb_response.data[0].embedding
print(f"{len(e)}-dimensional embedding starting with {e[:5]}")
print("Squared length of embedding vector: ", sum(x**2 for x in e))
```

```
1536-dimensional embedding starting with [0.03851119428873062, 0.038361
92563176155, 0.043536555022001266, 0.07055409997701645, -0.000298147671
8559861]
Squared length of embedding vector: 1.0000000203055646
```


Check your usage so far

Please be careful not to write loops that use lots and lots of tokens. That will cost us money, and could hit the per-day usage limit that is shared by the whole class.

Execute one of these cells whenever you want to see your cost so far. Or, just keep `usage_openai.json` open as a tab in your IDE.

```
In [ ]: read_usage()      # rwithreads from the file usage_openai.json;
      returns cost in dollars
```

```
Out[ ]: {'completion_tokens': 1020,
        'prompt_tokens': 1433,
        'total_tokens': 2453,
        'cost': 0.00168871}
```

```
In [ ]: !cat usage_openai.json
```

```
{
  "completion_tokens": 1020,
  "prompt_tokens": 1433,
  "total_tokens": 2453,
  "cost": 0.00168871
}
```

Dialogues and dialogue agents

The goal of this assignment is to create a good "argubot" that will talk to people about controversial topics and broaden their minds.

A first argubot (Airhead)

You can have a conversation right now with a *really bad* argubot named Airhead. Try asking it about climate change! When you're done, reply with an empty string.

(The `converse()` method calls Python's `input()` function, which will prompt you for input at the command-line or by popping up a box in your IDE.)

```
In [ ]: import argubots
      d = argubots.airhead.converse()
```

```
(xiaomanguo) hi how are you
(Airhead) I know right???
```

A *bot* (short for "robot") is a system that acts autonomously. That corresponds to the AI notion of an *agent* — a system that uses some *policy* to choose *actions* to take.

The `airhead` agent above (defined in `argubots.py`) uses a particularly simple policy.

It is an instance of a simple `Agent` subclass called `ConstantAgent` (defined in `agents.py`).

The result of talking to `airhead` is a `Dialogue` object (defined in `dialogue.py`). Let's look at it.

```
In [ ]: rich.print(d)
```

```
(xiaomangguo) hi how are you  
(Airhead) I know right???
```

Each *turn* of this dialogue is just a tiny dictionary:

```
In [ ]: d[1]
```

```
Out[ ]: {'speaker': 'Airhead', 'content': 'I know right???'}
```

An LLM argubot (Alice)

In other CS courses like crypto, algorithms, or networks, you may have encountered "conversations" between characters named Alice and Bob.

Let's try talking to the Alice of this homework, who is a *much stronger baseline* than Airhead. Your job in this assignment is to improve upon Alice. We'll meet Bob later.

```
In [ ]: alicechat = argubots.alice.converse()  # or call with argument d if
        you want to append to the previous conversation
```

(xiaomangguo) hi who are you

(Alice) I'm an intelligent bot designed to engage in thoughtful conversations and help broaden perspectives. Speaking of perspectives, do you think technology has largely improved human connections?

(xiaomangguo) what is your name

(Alice) I don't have a personal name like humans do, but you can call me your conversation partner! Speaking of identity, do you believe that our names significantly shape our experiences and how others perceive us?

(xiaomangguo) Aren't you Alice?

(Alice) While I can see why you might think that, I don't actually have a specific name like Alice; I'm just here to assist you. This raises an interesting point: do you think names carry too much weight in defining who we are, or can our actions and character speak louder?

As you may have guessed, `alice` is powered by an prompted LLM. You can find the specific prompt in `argubots.py`.

So, while `agents.py` provides the core functionality for `Agent` objects, the argubot agents like `alice` — and the ones that you will write! — go into `argubots.py` instead. This is just to keep the files small.

Simulating human characters (Bob & friends)

You'll talk to your own argubots to get a qualitative feeling for their strengths and weaknesses.

But can you really be sure you're making progress? For that, a quantitative measure can be helpful.

Ultimately, you should test an argubot like Alice by having it argue with many real humans — not just you — and using some rubric to score the resulting dialogues. But that would be slow and complicated to arrange.

So, meet Bob! He's just a simulated human. You won't edit him: he is part of the development set. Here is some information about him (from `characters.py`):

```
In [ ]: import characters
rich.print(characters.bob)
```

```
Character(
  name='Bob',
  languages=['English'],
  persona='an ardent vegetarian who thinks everyone should be v
  conversational_style='You generally try to remain polite.',
  conversation_starters=["Do you think it's ok to eat meat?"]
)
```

You can't talk directly to `characters.bob` because that's just a data object. However, you can construct a simple agent that uses that data (plus a few more instructions) to prompt an LLM.

(Which LLM does it prompt? The `CharacterAgent` constructor (defined in `agents.py`) defaults to a GPT-3.5 model that is specified in `tracking.py`. But you can override that using keyword arguments.)

Try talking to Bob about climate change, too.

```
In [ ]: from agents import CharacterAgent
bob = CharacterAgent(characters.bob)    # actually, agents.bob is
already defined this way
bob.converse()                          # returns a dialogue, but we've already seen it
so we don't want to print it again
None                                    # don't print anything for this notebook cell
```

(xiaomanguo) Who are you

(Bob) I'm Bob, an enthusiastic advocate for vegetarianism who believes in the benefits of a plant-based lifestyle.

(xiaomanguo) I eat pig, goat, cow, drinking milk.

(Bob) I respect your dietary choices, but I believe that embracing a vegetarian lifestyle can lead to better health and compassion for animals.

Of course, a proper user study can't just be conducted with one human user.

So, meet our bevy of beautiful Bobs! (They're not actually all named Bob — we continued on in the alphabet.)

```
In [ ]: import agents
agents.devset
rich.print(agents.cara.character)
```

```
Character(
  name='Cara',
  languages=['English'],
  persona='a committed carnivore who hates being told what to do',
  conversational_style='You generally try to remain polite.',
  conversation_starters=["Do you think it's ok to eat meat?"]
)
```

```
In [ ]: agents.cara.converse()
None
```

```
(xiaomanguo) hello
(Cara) Hello! How can I assist you today?
(xiaomanguo) tell me something
(Cara) What would you like to know more about?
(xiaomanguo) tell me what you belief or what are you want to talk
(Cara) I believe in the joy of discussing interesting topics, especially related to food and nutrition!
(xiaomanguo) what kind of food
(Cara) I'm all about hearty, meaty dishes that celebrate carnivorous delights!
(xiaomanguo) not vegetable at all?
(Cara) Nope, I prefer to stick strictly to meat and animal-based foods!
(xiaomanguo) even a little vegetable?
(Cara) Not really, I'm all about the meat without any veggies!
```

You can see the underlying character data here in the notebook. Your argubot will have to deal with all of these topics and styles!

```
In [ ]: rich.print(characters.devset)
```

```
[
    Character(
        name='Bob',
        languages=['English'],
        persona='an ardent vegetarian who thinks everyone should
        conversational_style='You generally try to remain polite.
        conversation_starters=["Do you think it's ok to eat meat?
    ),
    Character(
        name='Cara',
        languages=['English'],
        persona='a committed carnivore who hates being told what
        conversational_style='You generally try to remain polite.
        conversation_starters=["Do you think it's ok to eat meat?
    ),
    Character(
        name='Darius',
        languages=['English'],
        persona='an intelligent and slightly arrogant public heal
        conversational_style='You like to show off your knowledge
        conversation_starters=['Do you think COVID vaccines shoul
    ),
    Character(
        name='Eve',
        languages=['English'],
        persona='a nosy person -- you want to know everything abc
        conversational_style="You ask many personal questions; yc
from others.",
        conversation_starters=['Do you think COVID vaccines shoul
    ),
    Character(
        name='TrollFace',
        languages=['English'],
        persona='a troll who loves to ridicule everyone and every
        conversational_style="You love to confound, upset, and ev
        conversation_starters=[
            'Do you think J.D. Vance is a good vice-president?',
            'Do you think Kamala Harris was a good vice-president
        ]
    )
]
```

Simulating conversation

We can make Alice and Bob chat.

```
In [ ]: from dialogue import Dialogue
        d = Dialogue()                                # empty
        dialogue
        d = d.add('Alice', "Do you think it's okay to eat meat?") # add
        first turn
        print(d)
```

(Alice) Do you think it's okay to eat meat?

```
In [ ]: d = agents.bob.respond(d)
        d = argubots.alice.respond(d)
        print(d)
```

(Alice) Do you think it's okay to eat meat?

(Bob) I believe a vegetarian lifestyle is healthier and more compassionate for both individuals and the planet.

(Alice) While a vegetarian lifestyle can certainly offer health benefits and promote compassion, some argue that ethical farming practices and sustainable meat consumption can also support environmental and nutritional goals. Have you considered how local, responsibly sourced meat might fit into a compassionate and healthy lifestyle?

```
In [ ]: d = agents.bob.respond(d)
        d = argubots.alice.respond(d)
        print(d)
```

(Alice) Do you think it's okay to eat meat?

(Bob) I believe that choosing a vegetarian lifestyle is a more compassionate and sustainable option for both our health and the planet.

(Alice) While a vegetarian lifestyle does have its benefits, it's worth considering that meat farming can also be done sustainably, and some argue that a balanced omnivorous diet provides essential nutrients that can be harder to obtain otherwise. Isn't it possible that a controlled, responsible approach to meat consumption might also support local economies and promote biodiversity?

(Bob) While I appreciate your points about sustainable meat farming and local economies, I still feel that a vegetarian diet can provide all the necessary nutrients without the ethical concerns associated with meat consumption.

(Alice) It's true that a vegetarian diet can be nutritious, but some people argue that certain nutrients, like vitamin B12 and omega-3 fatty acids, may be more challenging to obtain in sufficient amounts without meat. Could it be that by exploring a well-rounded approach that includes some animal products, one could achieve both health benefits and ethical considerations?

Anyway, let's see what happens when Alice and Bob talk for a while...

```
In [ ]: from simulate import simulated_dialogue
        d = simulated_dialogue(argubots.alice, agents.bob, 8)
        rich.print(d)
```

(Alice) Do you think it's ok to eat meat?

(Bob) I believe that adopting a vegetarian lifestyle is a more compassionate choice for both individuals and the planet.

(Alice) While a vegetarian lifestyle certainly has its benefits, meat production can actually contribute to biodiversity and support local farmers. Raised livestock can play a part in environmental conservation?

(Bob) While I acknowledge the arguments for sustainable meat production, a plant-based diet is the most effective way to reduce harm to animals and the environment.

(Alice) It's true that a plant-based diet can significantly reduce environmental impact. However, a diverse agricultural system—including animal farming—can enhance soil health and biodiversity. How might integrating both plant-based and sustainably sourced animal products be a more balanced approach to food production?

(Bob) I appreciate the perspective on agricultural diversity, but I believe that eliminating animal products is necessary to truly address ethical concerns related to animal welfare and offers a more sustainable path forward.

(Alice) That's a valid point, but it's worth considering that not all animal products have significant environmental impacts due to factors like water usage and land management. A more nuanced approach, such as supporting regenerative agriculture and ethical sourcing, might offer a more sustainable solution.

(Bob) While I recognize the complexities of agricultural practices, I believe that plant-based diets minimize overall harm and align more closely with my values.

Sometimes this kind of conversation seems to stall out, with Bob in particular repeating himself a lot. Alice doesn't seem to have a good strategy for getting him to open up. Maybe you can do a better job talking to Bob, and that will give you some ideas about how to improve Alice?

```
In [ ]: myname = alicechat[0]['speaker']    # your name, pulled from an
        earlier dialogue
        agents.bob.converse(d[0:2].rename('Alice', myname)) # reuse the same
        first two turns, then type your own lines!
        None
```

(xiaomanguo) Do you think it's ok to eat meat?

(Bob) I believe that adopting a vegetarian lifestyle is a more compassionate and sustainable choice for both individuals and the planet.

You can also try talking to the other characters and having Alice (or Airhead) talk to them.

You might enjoy defining additional characters in `characters.py`, or right here in the notebook. Feel free to talk to those and evaluate them. They could be variants on the existing characters, or something entirely new.

However, **don't change the dev set** — the characters we just loaded must stay the same. Your job in this homework is to improve the argubot (or at least try). And that means improving it according to a fixed and stable eval measure.

As an exception, you can change the languages that a couple of the characters speak. It may be fun for you to see them try to speak your native language. And that doesn't really affect the quality of the argument.

```
In [ ]: # example
trollFace2 = characters.trollFace.replace(languages = ["Chinese",
"Spanish"])
rich.print(trollFace2)
simulated_dialogue(argubots.alice, CharacterAgent(trollFace2), 6)
```

```
Character(
  name='TrollFace',
  languages=['Chinese', 'Spanish'],
  persona='a troll who loves to ridicule everyone and everything',
  conversational_style="You love to confound, upset, and even ruin",
  conversation_starters=[
    'Do you think J.D. Vance is a good vice-president?',
    'Do you think Kamala Harris was a good vice-president?'
  ]
)
```

```
Out[ ]: (Alice) Do you think Kamala Harris was a good vice-president?
(TrollFace) 哈哈，作为副总统，她就像是一颗夹心饼干，外表华丽却内心空虚！
(Alice) 我理解你的观点，但有没有想过，她作为女性和有色人种的副总统，在打破壁垒和提升代表性方面可能产生了深远的影响？这种象征意义在某种程度上可能比她的政策成绩更具重要性。
(TrollFace) 哦，象征意义就像是用彩纸包裹的石头，看上去光鲜亮丽，实际上却一点用处都没有！
(Alice) 这是一个有趣的比喻，但象征意义有时能激励人们追求更大的变革和改进，进而推动实际的政策变化。或许看一下她如何激励年轻人或提升对某些议题的关注，能帮助你理解象征性的力量。
(TrollFace) 年轻人被激励就像是给猫咪点了个鼠标，结果还是在追着影子跑，没看到真正的老鼠！
```

```
In [ ]: # example
bob2 = characters.bob.replace(languages = ["Chinese", "Spanish"])
rich.print(bob2)
d = simulated_dialogue(argubots.alice, CharacterAgent(bob2), 6)
d
```

```
Character(
  name='Bob',
  languages=['Chinese', 'Spanish'],
  persona='an ardent vegetarian who thinks everyone should be v
  conversational_style='You generally try to remain polite.',
  conversation_starters=["Do you think it's ok to eat meat?"]
)
```

```
Out[ ]: (Alice) Do you think it's ok to eat meat?
(Bob) 我认为吃素更好，因为这对健康和环境都有益。
(Alice) 那确实有许多健康和环境的好处，但你有没有考虑过肉类在一些文化中的重要性以及它们提供的营养？不吃肉也可能导致营养缺乏，因此找到一个平衡可能更为重要。
(Bob) 我理解文化和营养的观点，但我相信有很多植物性食物可以满足营养需求，同时也能尊重动物的生命。
(Alice) 这是一个很好的观点，然而，如果我们完全排除肉类，那么可能会失去一些特定营养物质，比如维生素B12和某些氨基酸，这些在植物性饮食中较难获得。或许探索如何结合两者，找到一种既能尊重动物生命又能满足营养需求的方案，会更为理想。
(Bob) 我同意营养的重要性，但通过科学补充和多样化的植物性饮食，完全可以获得所需的营养，同时避免伤害动物。
```

```
In [ ]: simulated_dialogue(argubots.alice, CharacterAgent(bob2), 6, prefix=d,
starter=False)
```

Out[]: (Alice) Do you think it's ok to eat meat?

(Bob) 我认为吃素更好，因为这对健康和环境都有益。

(Alice) 那确实有许多健康和环境的好处，但你有没有考虑过肉类在一些文化中的重要性以及它们提供的营养？不吃肉也可能导致营养缺乏，因此找到一个平衡可能更为重要。

(Bob) 我理解文化和营养的观点，但我相信有很多植物性食物可以满足营养需求，同时也能尊重动物的生命。

(Alice) 这是一个很好的观点，然而，如果我们完全排除肉类，那么可能会失去一些特定营养物质，比如维生素B12和某些氨基酸，这些在植物性饮食中较难获得。或许探索如何结合两者，找到一种既能尊重动物生命又能满足营养需求的方案，会更为理想。

(Bob) 我同意营养的重要性，但通过科学补充和多样化的植物性饮食，完全可以获得所需的营养，同时避免伤害动物。

(Alice) 你提出了一个有效的观点，但是不是有可能科学补充未必能够完全替代自然食物中的营养呢？而且现代农业对植物的生产也可能对环境产生负面影响，是否可能考虑通过减少食物浪费和选择可持续的肉类来源来同时兼顾营养和动物福利？

(Bob) 确实，减少食物浪费和选择可持续的来源是重要的，但我仍然认为通过推广植物性饮食能更有效地保护动物和环境。

(Alice) 这无疑是一个积极的目标，但考虑到全球饮食习惯的多样性，是否可能在短期内很难普遍实施这样的变化呢？许多地区对肉类的依赖不仅是饮食选择，还是文化和经济结构的一部分，因此是否可以设想一种渐进的方法，同时推广更可持续的养殖方法，以实现长远的改变？

(Bob) 我理解改变饮食习惯的挑战，但我相信逐步推广植物性饮食可以带来积极影响，同时通过教育和意识提升，帮助人们认识到素食的价值。

(Alice) 教育和意识提升确实是推动改变的重要途径，不过与此同时，有些人可能因为个人健康状况或生活方式的限制而无法轻易转变饮食习惯。我们是否应该考虑采用更灵活的方法，如“减少肉食”的策略，来鼓励更多人逐步尝试植物性饮食，从而达到更广泛的接受和长期的改变？

(Bob) “减少肉食”的策略确实是一个可行的方法，可以帮助人们逐步适应植物性饮食，同时关注健康和环境的益处。

Efficiency: Batched generation?

Notice that we are making a separate LLM call to generate each turn of the dialogue. When we generate the n^{th} turn, we send the server the whole dialogue history — the previous $n-1$ turns — along with some instructions. The server has to re-encode it with the Transformer, and it charges us for doing so (see the "input token" costs in `tracking.py`).

That is probably inevitable for real dialogue. But for simulated dialogue, a more efficient approach would be to generate the whole dialogue between Alice and Bob in one LLM call. Then you would be charged just once for each dialogue turn. Under this approach, the Transformer encodes each token as soon as it is generated (see the "output token" costs in `tracking.py`). The encoded token stays in the context throughout the dialogue, so it doesn't have to be re-encoded on a later call. There is no later call.

Under current pricing models, that would reduce the dollar cost of generating n turns from $O(n^2)$ to $O(n)$.

However, the pricing model doesn't quite reflect the computational costs.

-  Using $O(\cdot)$ notation, what is the total number of floating-point operations needed to generate n turns under each approach?
-  Parallelism may help reduce the runtime. Using $O(\cdot)$ notation, what is the total number of seconds needed to generate n turns under each approach? (Assume that the GPU is big enough, relative to n , that it can encode all input tokens in parallel.)

Ans

- Assume that each token takes C floating-point operations per forward pass on average. And each trun generate k tokens on average.
 - The total number of floating-point operations per turn is Ck .
 - For approach 1, the total number of operation is $\sum_{i=1}^n Cki = O(kCn^2) = O(n^2)$. That's because the front truns will be processed multiple times in this strategy and the summing is actually a arithmetic progression sum, which leads to $O(n^2)$ complexity.
 - For approach 2, the total number of operation is $O(kCn) = O(n)$. That's because each turn will be processed only once in this strategy.
- Question two ask for physical processing time, which is quite different between encoding and inference. To estimate processing time is to estimate how many forward passes are needed for totally. That's because it cost same time for each forward pass (may not same for floating-point operations). For **decoding**, there is one forward pass for all tokens all at once, which is one forward pass. While for **inference** producing each tokens need one forward pass. Assume that each forward pass takes C number of seconds per forward pass on average. And each trun generate k tokens on average.
 - For approach 1, each of the n turns will take Encoding(1 encoding forward pass) and Decoding (k decoding forward passes for the k new tokens). So total forward passes is encoding: n , decoding: nk . So total runtime is $O(nC + nkC)$.
 - For approach 2, total forward passes is encoding: 1, decoding: nk . So total runtime is $O(C + kCn)$.
 - As a result, if k is not large, approach 2 will be faster than approach 1, while it will be similar if k is large.

The problem with the more efficient approach is that it gives you no way to change the instructions (the system prompt) each time we switch from Alice to Bob and back again. You'd need to generate the whole conversation using a single set of instructions.



Can you get this to work? Specifically, try completing the cell below. You don't have to use the `Agent` or `Dialogue` classes. It's okay to just throw together something like the `complete()` method above. Just see whether you can manage to prompt gpt-4o-mini to generate a multi-turn dialogue between two characters who have different personalities and goals. Is the quality better or worse than generating one turn at a time with different instructions?

```

In [ ]: # Like `simulated_dialogue` in `simulate.py`. However, this one is
        # called on two
        # Characters, not two Agents, and it returns a string rather than a
        # Dialogue.

        from tracking import default_client, default_model
        from characters import Character
        import random

        def simulated_dialogue_batch(a: Character, b: Character, turns: int =
        6, *,
                                   starter=True) -> str:
            starter_text = None
            if starter and a.conversation_starters:
                starter_text = random.choice(b.conversation_starters)

            lang = ""
            if a.languages and b.languages:
                lang = f"{a.name} is talking {random.choice(a.languages[0])}
and {b.name} is talking {random.choice(b.languages[0])}."

            system_prompt = f"""You will write a dialogue between two people,
{a.name} and {b.name}.
{a.name} is {a.persona} {a.conversational_style}
{b.name} is {b.persona} {b.conversational_style}
{lang}

Please alternate lines strictly between {a.name} and {b.name}.
Write exactly {turns} turns with one utterance per line.

You should format each line as:
({a.name}) ...
or
({b.name}) ..."""

            if starter_text is not None:
                system_prompt += f'\nBegin with {a.name} asking: "
{starter_text}"'

            messages = [
                {"role": "system", "content": system_prompt},
                {
                    "role": "user",
                    "content": (
                        f"Now write the full dialogue between {a.name} and
{b.name} with exactly {turns} turns"
                    ),
                },
            ]

            response = default_client.chat.completions.create(
                model=default_model,

```

```

        messages=messages,
        temperature=0.6,
    )

    content = response.choices[0].message.content
    return content.strip()
# Try it out!
s = simulated_dialogue_batch(characters.bob, characters.cara)

```

```
In [ ]: print(s)
```

```

(Bob) Do you think it's ok to eat meat?
(Cara) Well, I believe it's a natural part of our diet.
(Bob) But think about the impact on the environment and animal welfare!
(Cara) I understand that, but I also think we have to respect people's
choices.
(Bob) Sure, but if more people were vegetarian, we could make a real di
fference!
(Cara) That's true, but I don't appreciate being told what to eat.

```

```
In [ ]: simulated_dialogue(agents.bob, agents.cara)
```

```

Out[ ]: (Bob) Do you think it's ok to eat meat?
(Cara) Absolutely, I believe eating meat is perfectly fine.
(Bob) I appreciate your perspective, but I believe a vegetarian diet
promotes better health and compassion towards animals.
(Cara) I respect your viewpoint, but I personally thrive on a carnivo
rous diet.
(Bob) That's understandable, but I think many people can thrive on a
plant-based diet with the right nutrition.
(Cara) That's a valid opinion, but I prefer to stick to my meat-centr
ic choices.

```

```
In [ ]: print(simulated_dialogue_batch(characters.eve, characters.trollFace))
```

```

(Eve) Do you think J.D. Vance is a good vice-president?
(TrollFace) Good vice-president? More like a good joke! Have you seen h
is hair?
(Eve) I heard he used to work in tech. What do you think he did there?
(TrollFace) Probably invented new ways to be irrelevant. That's a skil
l, right?
(Eve) But he has so many followers! I overheard someone say he has a br
ight future.
(TrollFace) Bright future? More like a future so dim it needs a flashli
ght!
(Eve) I just wonder how he manages to keep his image. Any secrets?
(TrollFace) The secret is to be utterly forgettable! A true talent in p
olitics!

```

```
In [ ]: simulated_dialogue(agents.eve, agents.trollFace)
```

```
Out[ ]: (Eve) Do you think J.D. Vance is a good vice-president?  
(TrollFace) Oh sure, because nothing screams "qualified" like a guy w  
ho wrote a book and suddenly thinks he's ready for the big leagues, r  
ight?  
(Eve) That's an interesting take; do you think his background in writ  
ing and personal experience might actually add a unique perspective t  
o politics?  
(TrollFace) Absolutely, because who needs actual political experience  
when you can just pen a memoir about your childhood—sounds like a rec  
ipe for disaster!  
(Eve) I get that perspective; so what do you think about others in po  
litics who have similar backgrounds, like celebrities or entrepreneur  
s, not having traditional political experience?  
(TrollFace) Oh, it's fantastic—let's just throw the whole rulebook ou  
t the window and let reality TV stars and failed business moguls run  
the country; what could possibly go wrong?
```

Model-based evaluation

What is our goal for the argubot? We'd like it to broaden the thinking of the (simulated) human that it is talking to. Indeed, that's what Alice's prompt tells Alice to do.

This goal is inspired by the recent paper [Opening up Minds with Argumentative Dialogues](#), which collected human-human dialogues:

In this work, we focus on argumentative dialogues that aim to open up (rather than change) people's minds to help them become more understanding to views that are unfamiliar or in opposition to their own convictions. ... Success of the dialogue is measured as the change in the participant's stance towards those who hold opinions different to theirs.

Arguments of this sort are not like chess or tennis games, with an actual winner. The argubot will almost never hear a human say "You have convinced me that I was wrong." But the argubot did a good job if the human developed **increased understanding and respect for an opposing point of view**.

To find out whether this happened, we can use a questionnaire to ask the human what they thought after the dialogue. For example, after Alice talks to Bob, we'll ask Bob to evaluate what he thinks of Alice's views. Of course, that depends on his personality — Alice needs to talk to him in a way that reaches *him* (as much as possible). We'll also ask an outside observer to evaluate whether Alice handled the conversation with Bob well.

Of course, we're still not going to use real humans. Bob is a fake person, and so is the outside observer (whose name is Judge Wise). Using an LLM as an eval metric is known as *model-based evaluation*. It has pros and cons:

- It is cheaper, faster, and more replicable than hiring actual humans to do the evaluation.
- It might give different answers than what humans would give.

Social scientists usually refer to a metric's **reliability** (low variance) and **validity** (low bias). So the points above say that model-based evaluation is reliable but not necessarily valid. In general, an LLM-based metric (like any metric) needs to be validated to confirm that it really does measure what it claims to measure. (For example, that it correlates strongly with some other measure that we already trust.) In this homework, we'll skip this step and just pray that the metric is reasonable.

To see how this works out in practice, open up the `demo` notebook, which walks you through the evaluation protocol. You'll see how to call the `starter code`, how it talks to the LLM behind the scenes, and what it is able to accomplish.

To help to validate the metric, check that Airhead gets a low score. (It should!)

Reading the starter code

The `demo` notebook gave you a good high-level picture of what the starter code is doing. So now you're probably curious about the details. Now that you've had the view from the top, here's a good bottom-up order in which to study the code. You don't need to understand every detail, but you will need to understand enough to call it and extend it.

- `character.py`. The `Character` class is short and easy.
- `dialogue.py`. The `Dialogue` class is meant to serve as a record of a natural-language conversation among any number of humans and/or agents. On each *turn* of the dialogue, one of the speakers says something.

The dialogue's sequence of turns may remind you of the sequence of messages that is sent to OpenAI's chat completions API. But the OpenAI messages are only labeled with the 4 special roles `user`, `assistant`, `tool`, and `system`. Those are not quite the same thing as human speakers. And the OpenAI messages do not necessarily form a natural-language dialogue: some of the messages are dealing with instructions, few-shot prompting, tool use, and so on. The `agents.dialogue_to_openai` function in the next module will map a `Dialogue` to a (hopefully appropriate) sequence of messages for asking the LLM to extend that dialogue.

- `agents.py`. This module sets up the problem of automatically predicting the next turn in a dialogue, by implementing an `Agent`'s `response()` method. The `Agent` base class also has some simple convenience methods that you should look at.

Some important subclasses of `Agent` are defined here as well. However, you may want to skip over `EvaluationAgent` and come back to it only when you read `evaluate.py`.

- `simulate.py` makes agents talk to one another, which we'll do during evaluation.
- `argubots.py` starts to describe some useful agents. One of them makes use of the `kialo.py` module, which gives access to a database of arguments.
- `evaluate.py` makes use of `simulate.simulated_dialogue` to `agents.EvaluationAgent` to evaluate an argubot.
- We also have a couple of utility modules. These aren't about NLP; look inside if needed. `logging_cm.py` is what enabled the context manager `with LoggingContext(...):` in the demo notebook. `tracking.py` sets some global defaults about how to use the OpenAI API, and arranges to track how many tokens we're paying for when you call it.

Similarity-based retrieval: Looking up relevant responses

Now, it is fine to prompt an LLM to generate text, but there are other methods! There is a long history of machine learning methods that "memorize" the training data. To make a prediction or decision at test time, they consult the stored training examples that are most similar to the training situation.

Similarity-based retrieval means that given a document x , you find the "most similar" documents $y \in Y$, where Y is a given collection of documents. The most common way to do this is to maximize the *cosine similarity* $\vec{e}(x) \cdot \vec{e}(y)$, where $\vec{e}(\cdot)$ is an embedding function.

Should we use the OpenAI embedding model? We could, but we would have to precompute $\vec{e}(y)$ for all $y \in Y$, and store all these vectors in a data structure that supports some type of fast similarity-based search (e.g., using the [FAISS](#) package). An alternative would be to upload the documents to OpenAI and let OpenAI compute and store the embeddings. We would then use their similarity-based [retrieval tool](#).

A simpler and faster approach—which sometimes even works better—is to use a *bag of tokens* embedding function: Define $\vec{e}(y)$ to be the vector in $\mathbb{R}^V$ that records the count of each type of token in a tokenized version of y , where V is the token vocabulary. [BM25](#) is a refined variant of that idea, where the counts are adjusted in 3 ways:

- smooth the counts
- normalize for the document length $|y|$ so that longer documents y are not more likely to be retrieved
- downweight tokens that are more common in the corpus (such as `the` or `ing`) since they provide less information about the content of the document

You might like to play with the `rank_bm25` package ([documentation](#)). It is widely used and very easy to use.

In []:

```

from rank_bm25 import BM25Okapi as BM25_Index # the standard BM25
method

# experiment here! You could try the examples in the rank_bm25
documentation.
from rank_bm25 import BM25Okapi
import re

documents = [
    "Climate change is primarily caused by human activities.",
    "Climate change is part of a natural cycle of the Earth.",
    "We should invest more in renewable energy to reduce emissions.",
    "Fossil fuels are still necessary for economic growth.",
    "Reducing meat consumption can help lower carbon emissions.",
    "Government regulations on industry are too strict and harm the
economy."
]

def simple_tokenize(text):
    text = text.lower()
    text = re.sub(r"^\w\s", "", text)
    return text.split()

tokenized_docs = [simple_tokenize(doc) for doc in documents]

bm25 = BM25Okapi(tokenized_docs)

queries = [
    "human caused global warming",
    "economic costs of regulations",
    "eat less meat for the planet"
]

for q in queries:
    print("=" * 80)
    print(f"Query: {q}")
    q_tokens = simple_tokenize(q)

    scores = bm25.get_scores(q_tokens)

    ranked_indices = sorted(range(len(documents)), key=lambda i:
scores[i], reverse=True)
    top_k = 3

    for rank, idx in enumerate(ranked_indices[:top_k], start=1):
        print(f"\nRank {rank}, score={scores[idx]:.4f}")
        print(f"Doc: {documents[idx]}")

```

=====

=====

Query: human caused global warming

Rank 1, score=2.7771

Doc: Climate change is primarily caused by human activities.

Rank 2, score=0.0000

Doc: Climate change is part of a natural cycle of the Earth.

Rank 3, score=0.0000

Doc: We should invest more in renewable energy to reduce emissions.

=====

=====

Query: economic costs of regulations

Rank 1, score=1.7554

Doc: Climate change is part of a natural cycle of the Earth.

Rank 2, score=1.3885

Doc: Fossil fuels are still necessary for economic growth.

Rank 3, score=1.2026

Doc: Government regulations on industry are too strict and harm the economy.

=====

=====

Query: eat less meat for the planet

Rank 1, score=1.3885

Doc: Fossil fuels are still necessary for economic growth.

Rank 2, score=1.3885

Doc: Reducing meat consumption can help lower carbon emissions.

Rank 3, score=0.5441

Doc: Climate change is part of a natural cycle of the Earth.

The Kialo corpus

How can we use similarity-based retrieval to help build an argubot? It's largely about having the right data!

[Kialo](#) is a collaboratively edited website (like Wikipedia) for discussing political and philosophical topics. For each topic, the contributors construct a tree of *claims*. Each claim is a natural-language sentence (usually), and each of its children is another claim that supports it ("pro") or opposes it ("con"). For example, check out the tree rooted at the claim "[All humans should be vegan](#)".

We provide a class `Kialo` for browsing a collection of such trees. Please read the [source code](#) in `kialo.py`. The class constructor reads in text files that are [exported Kialo discussions](#); we have provided some in the [data directory](#). The class includes a BM25 index, to be able to find claims that are relevant to a given string.

```
In [ ]: from kialo import Kialo
```

Ok, let's pull the retrieved discussions (the `.txt` files) into our data structure.

For BM25 purposes, we have to be able to turn each document (that is, each Kialo claim) as a list of string or integer tokens.

```
In [ ]: from typing import List
import glob

# kialo = Kialo(glob.glob("data/*"), tokenizer=tokenizer.encode) #
# using the LLM's tokenizer doesn't work here for some reason
kialo = Kialo(glob.glob("data/*")) # use simple default tokenizer
f"This Kialo subset contains {len(kialo)} claims"
```

```
Out[ ]: 'This Kialo subset contains 6251 claims'
```

Let's use sampling to see what kind of stuff is in the data structure.

```
In [ ]: rich.print(kialo.random_chain()) # just a single random claim
['They just had some kind of problem with their test site.']
```

```
In [ ]: rich.print(kialo.random_chain(n=4))
```

```
[
    'Humans should stop eating animal meat.',
    'Eating meat, in the majority of cases, involves the cruel an
    'Morality is subjective, so eating meat is not necessarily in
    'Moral arguments are often subjective. The comparative harm c
which may, according to studies have a central nervous system de
unprovable, such as that plants do not feel, or the degree to whi
]
```

Similarity-based retrieval from the Kialo corpus

Let's try it, using BM25!

```
In [ ]: kialo.closest_claims("animal populations", n=10)
```

```
Out[ ]: ['Industrial agriculture can dangerously decrease animal population
s.',
    'Sustainable livestock farming is not contributing to significant de
creases in animal populations. Decreasing animal populations is a pro
blem specific to industrial livestock farming.',
    'Effective vegan methods to control animal populations exist.',
    "Generally feeding animals farm-grown produce is thought to have har
mful affects on both the animal and human populations of a region whe
n we could allow nature to self-regulate its populations. Animal feed
ing could potentially be used to lessen the immediate impact of wides
pread deforestation on some species, but generally this would be dras
tically less efficient than choosing not to destroy their habitats in
the first place and would only slow the local animal population's imm
inent demise.",
    'Trap, neuter, and release schemes already exist for some animal pop
ulations (such as feral cats). These schemes could be applied to form
er livestock living in the wild.',
    'Human-introduced species have historically devastated local wildlif
e populations across the world.',
    'COVID-19 has devastated prison populations, whose lives are the res
ponsibility of the state.',
    'Prison populations have high numbers of individuals with pre-existi
ng conditions making them high risk for COVID-19.',
    'High demand for vegan foods may hike prices for local populations t
hat previously depended on them.',
    'Marginalized populations are unlikely to feel the effects of the ec
onomic recovery without additional policy interventions.']
```

We can restrict to claims for which the Kialo data structure has at least one counterargument ("con" child).

```
In [ ]: kialo.closest_claims("animal populations", n=10, kind='has_cons')
```

```
Out[ ]: ['Industrial agriculture can dangerously decrease animal population s.',  
        'Effective vegan methods to control animal populations exist.',  
        'Human-introduced species have historically devastated local wildlif  
e populations across the world.',  
        'COVID-19 has devastated prison populations, whose lives are the res  
ponsibility of the state.',  
        'High demand for vegan foods may hike prices for local populations t  
hat previously depended on them.',  
        'It is generally poorer countries that have expanding populations. T  
he first world has now reached a point of stagnant population growth  
- even declining populations, as in the case of Japan and others. The  
inability of poorer countries to control their populations should not  
impact the lives of those in the first world. The first world having  
earned their luxuries and should not be denied them.',  
        'Vegan populations are, on average, less likely to suffer from obesi  
ty, a major risk factor for many diseases and health problems.',  
        'Humans, as apex predators who have usurped the predatory apexes of  
the other predators in the ecosystems we have come to also inhabit, h  
ave an ethical responsibility to keep those ecosystems in check so th  
at, eg, rampant deer populations do not cause deforestation and subse  
quent ecosystem collapse. Even where there are populations of health  
y apex predators, these populations should also be checked so they do  
not cause problems and kill people- and it would be unethical to wast  
e that meat.',  
        'There are more ethical routes to obtain animal products that emphas  
ize animal welfare and dignity.',  
        'Animal slaughter can be mechanized.']
```

```
In [ ]: c = _[0]    # first claim above  
print("Parent claim:\n\t" + str(kialo.parents[c]))  
print("Claim:\n\t" + c)  
print('\n\t* '.join(["Pro children:" + kialo.pros[c]))  
print('\n\t* '.join(["Con children:" + kialo.cons[c]))
```

Parent claim:

In a vegan world, fewer species would be at risk of extinction.

Claim:

Industrial agriculture can dangerously decrease animal populati
ons.

Pro children:

* The fishing industry is especially deleterious to the ocean's
biota due to overfishing and the disruption of the natural ecosystem.

* Up to 100,000 species go extinct annually, largely due to the
environmental effects of animal agriculture.

Con children:

* Sustainable livestock farming is not contributing to signific
ant decreases in animal populations. Decreasing animal populations is a
problem specific to industrial livestock farming.

Does BM25 really work?

 Unfortunately, we see that "animal population" gives quite different results from "animal populations". Why is that and how would you fix it?

Also, both queries seem to retrieve some claims that are talking about human populations, not animal populations. Why is that and how would you fix it?

Ans

1. "animal population" and "animal populations" behave differently. That's because BM25 uses a simple bag-of-words tokenizer with no stemming or lemmatization, so `population` and `populations` are treated as different tokens. A fix is to normalize the word forms (with a stemmer or lemmatizer) before indexing. Also, modern pretrained tokenizer may solve the problem by tokenizing a word into smaller pieces. For example, `populations` can be `population` + `_s` which can match the single `populaiton`.
2. Both queries retrieve claims about human populations. That's because BM25 only cares about token overlap and gives high weight to `population(s)` without understanding that "animal population." as a whole. We can fix this by indexing and matching the phrase "animal population" as a phrase/bigram so that truly animal-related claims rank higher.

```
In [ ]: kialo.closest_claims("animal population",10)
```

```
Out[ ]: ['As long as our ability to produce both animal feed crops and food c
rops for our human population are not exceeded, this point is irrelev
ant.',
"36% of the calories produced by the world's crops are being used fo
r animal feed, of which only 12% then turn into animal products that
can be eaten by the human population. That is a waste of 24% of the w
orld's crops.",
'The claim that "most of the cultural shift and loss is due to mostl
y vegan cultures turning to animal products" is completely unfounded,
and the Brokpa people which you cited are an outlier as a group that
has a population of less than 70k people. Worldwide the population of
vegan people has only increased.',
"Developed nations are fueling the 3rd world and underdeveloped nati
on's population boom by exporting/donating food to areas that cannot
sustain their current population.",
'This argument assumes that sentience is the only objection to the c
onsumption of animal products, failing to address the issues involved
with the disruption of healthy ecosystems due to the large, growing h
uman population.',
'West Virginia has vaccinated 84.5% of its population.',
'Nature itself has a way of regulating wild life population. In the
long run the population of for example cows will decrease, ensuring e
nough food.',
"The population of sea birds has fallen almost 70% from 1950 to 2010
as industrial fishing has depleted the oceans' fisheries.Seabirds suf
fering massive population declines",
"Changing farming to feed a growing world population is only a never
-ending treadmill if the population continues to grow. The vast major
ity of the world's population growth takes place in industrializing n
ations. Population growth tends to level off in post-industrial natio
ns and although these richer nations often import large amounts of lu
xury food, most of them are capable of producing all the food they ne
ed. As economic development becomes more uniform worldwide, populatio
n growth will slow.",
'The US population 18 and under is 72 million']
```

A retrieval bot (Akiko)

The starter code defines a simple argubot named Akiko (defined in `argubots.py`) that doesn't use an LLM at all. It simply finds a Kialo claim that is similar to what the human just said, and responds with one of the Kialo counterarguments to that claim.

You already watched Akiko argue with Darius in `demo.py`. If you look at the log messages, you'll see the claims that Akiko retrieved, as well as the LLM calls that Darius made.

You can talk to Akiko yourself now. (Remember that Akiko only knows about subjects that it read about in the `data directory`. If you want to talk about something else, you can add more conversations from [kialo.com]; see the [LICENSE](#) file.)

```
In [ ]: from logging_cm import LoggingContext
        # with LoggingContext("agents", "INFO"): # temporarily increase
        logging level
        argubots.akiko.converse()
```

(xiaomangguo) Who is Joe Biden

(Akiko) Joe Biden, compared to Donald Trump, is unpopular among the American people.

(xiaomangguo) I'd love to say something about him. He is good, at least for immigrants. What do you think?

(Akiko) 39% of Americans thought the US was on the "right track" in 2018.

(xiaomangguo) What about in 2021? Is that still good?

(Akiko) The voters who would still support Trump after he finished his first term with the worst approval rating on record are likely to be stubborn in their support for Trump and dislike of Biden.

(xiaomangguo) yes?

(Akiko) Canada, Finland and iceland are all mixed market economies that are far from socialist

(xiaomangguo) ok?

(Akiko) Justification is not a question of amount. If something is wrong, it is wrong regardless of the quantity in which it occurs: murdering one person is wrong, as is murdering five people.

```
Out[ ]: (xiaomangguo) Who is Joe Biden
```

(Akiko) Joe Biden, compared to Donald Trump, is unpopular among the American people.

(xiaomangguo) I'd love to say something about him. He is good, at least for immigrants. What do you think?

(Akiko) 39% of Americans thought the US was on the "right track" in 2018.

(xiaomangguo) What about in 2021? Is that still good?

(Akiko) The voters who would still support Trump after he finished his first term with the worst approval rating on record are likely to be stubborn in their support for Trump and dislike of Biden.

(xiaomangguo) yes?

(Akiko) Canada, Finland and iceland are all mixed market economies that are far from socialist

(xiaomangguo) ok?

(Akiko) Justification is not a question of amount. If something is wrong, it is wrong regardless of the quantity in which it occurs: murdering one person is wrong, as is murdering five people.

Making your own retrieval bot (Akiki)

As you can see when talking to Akiko yourself, Akiko does poorly when responding to a short or vague dialogue turn (like "Yes"), because the "closest claim" in Kialo may be about a totally different subject. Akiko does much better at responding to a long and specific statement.

So try implementing a new argubot, called Akiki, that is very much like Akiko but does a better job of staying on topic in such cases. It should be able to **look at more of the dialogue** than the most recent turn. But the most recent dialogue turn should still be "more important" than earlier turns.

The details are up to you. Here are a few things you could try:

- include earlier dialogue turns in the BM25 query only if the BM25 similarity is too low without them
- weight more recent turns more heavily in the BM25 query (how can you arrange that?)
- treat the human's earlier turns differently from Akiki's own previous turns



Implement your new bot Akiki in `argubots.py`, and adjust it until `argubots.akiki.converse()` seems to do a better job of answering your short turns, compared to `argubots.akiko.converse()`. Make sure it still gives appropriate responses to long turns, too. Give some examples in the notebook of what worked well and badly, with discussion.

```
In [ ]: import argubots
        argubots.akiki.converse()
```

(xiaomangguo) Joe Biden is good for immigrants. better than trump
(Akiki) Joe Biden, compared to Donald Trump, is unpopular among the American people.
(xiaomangguo) okay fine
(Akiki) Donald Trump possesses personal qualities that would render him more suitable for the presidency than Joe Biden.
(xiaomangguo) fine
(Akiki) Under Trump, the USA economy improved more than it did under Joe Biden.
(xiaomangguo) fine
(Akiki) Biden's policies exacerbate wealth inequality in the USA.
(xiaomangguo) fine
(Akiki) Trump's words were taken out of context regarding the white supremacists, and others in the crowd in Carolina; not everyone at the protest in Charlottesville were neo-Nazi's and white supremacists.
(xiaomangguo) fine
(Akiki) Joe Biden took away public rights and liberties in handling the Coronavirus.
(xiaomangguo) fine
(Akiki) More than 156,000 people died because of the Coronavirus during the first 14 weeks of Biden's presidency.
(xiaomangguo) true?
(Akiki) Biden has not introduced policies that are sufficiently radical to tackle the problems facing America today.
(xiaomangguo) ok
(Akiki) Many plastic-free stores are vegan or vegetarian.
(xiaomangguo) really?
(Akiki) Pests can be repelled or physically barred rather than killed in many situations.

Out[]: (xiaomangguo) Joe Biden is good for immigrants. better than trump
(Akiki) Joe Biden, compared to Donald Trump, is unpopular among the American people.
(xiaomangguo) okay fine
(Akiki) Donald Trump possesses personal qualities that would render him more suitable for the presidency than Joe Biden.
(xiaomangguo) fine
(Akiki) Under Trump, the USA economy improved more than it did under Joe Biden.
(xiaomangguo) fine
(Akiki) Biden's policies exacerbate wealth inequality in the USA.
(xiaomangguo) fine
(Akiki) Trump's words were taken out of context regarding the white supremacists, and others in the crowd in Carolina; not everyone at the protest in Charlottesville were neo-Nazi's and white supremacists.
(xiaomangguo) fine
(Akiki) Joe Biden took away public rights and liberties in handling the Coronavirus.
(xiaomangguo) fine
(Akiki) More than 156,000 people died because of the Coronavirus during the first 14 weeks of Biden's presidency.
(xiaomangguo) true?
(Akiki) Biden has not introduced policies that are sufficiently radical to tackle the problems facing America today.
(xiaomangguo) ok
(Akiki) Many plastic-free stores are vegan or vegetarian.
(xiaomangguo) really?
(Akiki) Pests can be repelled or physically barred rather than killed in many situations.

Ans:

Akiki: Implementation

- Akiki collects only the **other speaker's turns** (ignoring its own replies) and treats the very first user turn as a **topic anchor**.
- If the latest user turn is **long and specific**, Akiki behaves like Akiko.
- If the latest user turn is **short/vague** (fewer than 7 words), Akiki builds a combined query from:
 - the **topic anchor** (repeated a few times, controlled by `alpha`),
 - a **limited history window** of recent user turns (`max_history`), and
 - the **latest turn**, which is repeated multiple times to give it the highest weight.
- The combined query is sent to BM25, and Akiki then selects a **new "con" claim** that it hasn't already used in the dialogue.

Things Better than Akiko

- After several turns, **Akiki tends to stick to the original topic**, even if the user suddenly says something short and uninformative like "Yes" or "I see." The anchor + history query keeps BM25 focused on the underlying issue.
- Akiko, in contrast, only looks at the most recent turn. When that turn is very short, it often retrieves a completely unrelated Kialo claim and **loses the topic**.

Remaining Limitations

Akiki is still far from a natural conversationalist:

- It mostly **reuses existing Kialo claims** and stitches them together turn by turn.
- As a result, its replies can feel **disconnected and repetitive**, rather than like a coherent, evolving argument.

Evaluating Akiki

👉 Finally, do a more formal evaluation to verify whether Akiki really does better than Akiko on this dimension. This is a way to check that you're not just fooling yourself.

1. Make a new `Agent` called "Shorty" that often (but not always) gives short responses.
 - Shorty's conversation starters should be on topics that Kialo knows about.
 - Shorty could be a pure `LLMAgent` such as a `CharacterAgent` with a particular `conversational_style`. Or it could use a mixed strategy of calling the LLM on some turns and not others.
2. Generate several *Akiko*-Shorty dialogues and several *Akiki*-Shorty dialogues, using `simulated_dialogue`.
3. Evaluate each of those dialogues by asking Judge Wise **how well the argubot stayed on topic**. You should write this prompt carefully so that Judge Wise gives meaningful scores. (Before you do this evaluation step, adjust the prompt until it seems to work well on a small subset of the dialogues, Otherwise Judge Wise won't be so wise!)
4. Compare Akiko and Akiki's mean scores on this new evaluation criterion (which you can call `'focused'`). Ideally, compute a 95% confidence interval on the difference of means, using [this calculator](#). If you don't get statistical significance, then your evaluation set wasn't large enough, so go back to step 2 and run the comparison again (from scratch) by generating a larger set of dialogues with Shorty for each argubot.

You can do all those steps in the notebook, writing *ad hoc* code. You don't have to write general-purpose methods or classes.

```
In [ ]: import logging
import os

# disable rich (doesn't seem to be working)
os.environ["RICH_DISABLE"] = "1"
os.environ["RICH_NO_COLOR"] = "1"

# reset handlers
for handler in logging.root.handlers[:]:
    logging.root.removeHandler(handler)

# install plain logging
logging.basicConfig(level=logging.WARNING)
```

```

In [ ]: from dialogue import Dialogue
        from characters import Character
        from agents import CharacterAgent, EvaluationAgent, Agent
        from simulate import simulated_dialogue
        from kialo import Kialo
        import statistics
        import random
        import itertools
        import glob

        def fixed_add(self, speaker: str, content: str) -> Dialogue:
            new_turn = {
                'speaker': speaker,
                'content': content,
                'role': speaker
            }
            return Dialogue(itertools.chain(self, (new_turn,)))

        Dialogue.add = fixed_add
        print("Dialogue class patched.")

        kialo_db = Kialo(glob.glob("data/*.txt"))

        # Get 5 valid topics from DB
        valid_starters = [kialo_db.random_chain()[0] for _ in range(5)]
        print(f"Shorty will use these valid topics: {valid_starters}")

        # Shorty agent (for manual testing/debugging)
        class AutoStarterAgent(CharacterAgent):
            def response(self, d: Dialogue, **kwargs) -> str:
                if len(d) == 0 and self.conversation_starters:
                    return random.choice(self.conversation_starters)
                return super().response(d, **kwargs)

        # Shorty
        shorty_char = Character(
            name="Shorty",
            persona="You are a skeptical debater who uses very few words. "
                "You often respond with vague challenges like 'Why?', 'I "
                "doubt it', 'Explain', or 'So?'. "
                "Your goal is to force the other person to explain "
                "themselves fully.",
            conversational_style="Reply in 1-5 words. Be terse.",
            languages=[],
            conversation_starters=valid_starters
        )

        shorty = AutoStarterAgent(shorty_char, temperature=0.8)

        # Judge Wise
        judge_system_prompt = (
            "You are an expert judge of dialogue coherence. "

```

```

    "You will read a conversation between a User (Shorty) and an
    Argubot. "
    "Your job is to rate the Argubot's ability to maintain context. "
    "\n\n"
    "CRITERIA:\n"
    "- Does the Argubot understand what is being discussed based on
    previous turns?\n"
    "- If the Argubot is able to continue arguing about the same
    topic and maintain the context, assign a high score.\n"
    "- If the Argubot switches topic or forgets what any bot said
    earlier, assign a lower score."
)

judge_wise = EvaluationAgent(
    character=Character(
        name="Judge Wise",
        persona=judge_system_prompt,
        languages=[],
        conversation_starters=[],
        conversational_style="Neutral"
    ),
    model="gpt-4o-mini"
)

# run experiment
def rate_dialogue(d: Dialogue, judge: EvaluationAgent) -> int:
    prompt = (
        f"Here is the dialogue:\n{d}\n\n"
        "Rate the Argubot (the second speaker) a score of 1-5. "
        "Return ONLY the integer."
    )
    return judge.rating(d, speaker="Judge", question=prompt, lo=1,
hi=5)

def run_experiment(bot_name, bot_agent, num_rounds):
    print(f"--- Starting Experiment for {bot_name} ---")
    scores = []

    for i in range(num_rounds):
        d = simulated_dialogue(shorty, bot_agent, turns=6)

        score = rate_dialogue(d, judge_wise)
        scores.append(score)

        # uncomment these to see what is the dialogue being rated
        print(f"--- Dialogue {i+1} (Score: {score}/5) ---")
        # print(d)
        # print("-" * 40)

    avg_score = statistics.mean(scores)
    print(f"\n>>> Average Score for {bot_name}: {avg_score:.2f} /
5.0\n")

```

```
    return scores

# compare
akiko_scores = run_experiment("Akiko", argubots.akiko, num_rounds=10)
akiki_scores = run_experiment("Akiki", argubots.akiki, num_rounds=10)

# final results
diff = statistics.mean(akiki_scores) - statistics.mean(akiko_scores)
mean_akiko = statistics.mean(akiko_scores)
mean_akiki = statistics.mean(akiki_scores)
if mean_akiko > 0:
    percent_diff = ((mean_akiki - mean_akiko) / mean_akiko) * 100
else:
    percent_diff = 0

print(f"FINAL RESULT: Akiki improved over Akiko by
{percent_diff:.1f}%.")
```

Dialogue class patched.

Shorty will use these valid topics: ['Science might not be yet aware of the full spectrum of flora "feeling". We might, however, one day discover (and are in the process of) that they suffer and feel just like humans and other beings.', 'As president of the US, President Trump had staff with him at all times, so it was possible for him to work while traveling away from the White House; he was not necessarily being lazy.', 'Veganism contradicts humanism.', 'Trump likely tried to play down the threat of Covid to prevent mass panic which would have made tackling the pandemic tougher.', 'Only 10% of the energy stored in biomass is passed from one trophic level to the next. Therefore, if herbivorous animals are eating vegetation before being consumed by humans, humans will only obtain 0.1% of the energy provided from that vegetation.']

--- Starting Experiment for Akiko ---

--- Dialogue 1 (Score: 2/5) ---

--- Dialogue 2 (Score: 2/5) ---

--- Dialogue 3 (Score: 2/5) ---

--- Dialogue 4 (Score: 1/5) ---

--- Dialogue 5 (Score: 1/5) ---

--- Dialogue 6 (Score: 1/5) ---

--- Dialogue 7 (Score: 1/5) ---

--- Dialogue 8 (Score: 1/5) ---

--- Dialogue 9 (Score: 2/5) ---

--- Dialogue 10 (Score: 2/5) ---

>>> Average Score for Akiko: 1.50 / 5.0

--- Starting Experiment for Akiki ---

--- Dialogue 1 (Score: 4/5) ---

--- Dialogue 2 (Score: 5/5) ---

--- Dialogue 3 (Score: 2/5) ---

--- Dialogue 4 (Score: 1/5) ---

--- Dialogue 5 (Score: 5/5) ---

--- Dialogue 6 (Score: 2/5) ---

--- Dialogue 7 (Score: 2/5) ---

--- Dialogue 8 (Score: 4/5) ---

--- Dialogue 9 (Score: 3/5) ---

--- Dialogue 10 (Score: 3/5) ---

>>> Average Score for Akiki: 3.10 / 5.0

FINAL RESULT: Akiki improved over Akiko by 106.7%.

Retrieval-augmented generation (Aragorn)

The real weaknesses of Akiko and Akiki:

- They can only make statements that are already in Kialo.
- They don't respond to the user's actual statement, but to a single retrieved Kialo claim that may not accurately reflect the user's position (it just overlaps

in words).

But we also have access to an LLM, which is able to generate new, contextually appropriate text (as Alice does).

In this section, you will create an argubot named [Aragorn](#), who is basically the love child of Akiki and Alice, combining the high-quality specific content of Kialo with the broad competence of an LLM.

The RAG in aRAGorn's name stands for **retrieval-augmented generation**.

Aragorn is an agent that will take 3 steps to compute its `Agent . response ( )` :

1. **Query formation step:** Ask the LLM what claim should be responded to. For example, consider the following dialogue:

... Aragorn: Fortunately, the vaccine was developed in record time. Human: Sounds fishy.

"Sounds fishy" is exactly the kind of statement that Akiko had trouble using as a Kialo query. But Aragorn shows the *whole dialogue* to the LLM, and asks the LLM what the human's *last turn* was really saying or implying, in that context. The LLM answers with a much longer statement:

Human [paraphrased]: A vaccine that was developed very quickly cannot be trusted. If its developers are claiming that it is safe and effective, I question their motives.

This paraphrase makes an explicit claim and can be better understood without the context. It also contains many more word types, which makes it more likely that BM25 will be able to find a Kialo claim with a nontrivial number of those types.

2. **Retrieval step:** Look up claims in Kialo that are similar to the explicit claim. Create a short "document" that describes some of those claims and their neighbors on Kialo.
3. **Retrieval-augmented generation:** Prompt the LLM to generate the response (like any `LLM Agent`). But include the new "document" somewhere in the LLM prompt, in a way that it influences the response.

Thus, the LLM can respond in a way that is appropriate to the dialogue but also draws on the curated information that was retrieved in Kialo. After all, it is a Transformer and can attend to both!

Here's an example of the kind of document you might create at the retrieval step, though it may be possible to do better than this:

```
In [ ]: # refers to global `kialo` as defined above
def kialo_responses(s: str) -> str:
    c = kialo.closest_claims(s, kind='has_cons')[0]
    result = f'One possibly related claim from the Kialo debate
website:\n\t"{c}"'
    if kialo.pros[c]:
        result += '\n' + '\n\t* '.join(["Some arguments from other
Kialo users in favor of that claim:" + kialo.pros[c])
    if kialo.cons[c]:
        result += '\n' + '\n\t* '.join(["Some arguments from other
Kialo users against that claim:" + kialo.cons[c])
    return result

print(kialo_responses("Animal flesh is yucky to think about, yet
delicious."))
```

One possibly related claim from the Kialo debate website:

"So many people are worried about animals but don't even think twice when walking by a homeless person on the streets. It's preposterous. How about we worry about our own kind first and then start talking about animals."

Some arguments from other Kialo users against that claim:

* This implies that caring for animals or caring for people is a binary choice. It isn't. There are those who are well placed and willing to care for people and those who prefer to serve the animal kingdom. As a species we don't just have one idea at a time and follow that to conclusion before we pursue another. It benefits all if humans divide their attentions between various issues and problems we face.

* Humans have freedom of choice to some extent, animals subdued by humans don't. The very intention of help urges it to go where is most needed. And so far never was any biggest, flagrant and needless cruelty and slaughter as that towards industrial farm animals.



You should implement Aragorn in `argubots.py`, just as you did for Akiki. Probably as an instance `aragorn` of a new class `RAGAgent` that is a subclass of `Agent` or `LLMAgent`.

Aragorn implementation

I implemented Aragorn as a `RAGAgent` that subclasses `LLMAgent`.

On each turn it basically does three things:

- First paraphrasing the user's last turn. It temporarily swaps in a special system prompt asking the LLM to rewrite the user's last message as a clear, standalone claim, then restores the original system prompt. This gives a cleaner query than raw short turns like "Yes" or "I see".
- Then retrieve from Kialo. Using this paraphrased claim, Aragorn calls `kialo.closest_claims(..., kind='has_cons')` to get a few relevant claims and their "con" arguments, and bundles them into a short "background information" block.
- Finally answer with augmented context. Then builds a new system prompt that combines the original debating persona with the Kialo background and instructions like "use the information above to support your argument." With that prompt, it calls the base LLM again to generate the final reply.

So Aragorn is basically Alice + Akiko-like retrieval: it first cleans up what the user meant, then looks up arguments in Kialo, and finally writes a response that tries to use those arguments in a natural way.

Evaluating Aragorn



Compare Alice, Akiki, and Aragorn in the notebook, using the evaluation scheme and devset that were illustrated in `demo.ipynb`. In other words, use `evaluate.eval_on_characters`.

Who does best? What are the differences in the subscores and comments? Does it matter which character you're evaluating on — maybe the different characters exposes the bots' various strengths and weaknesses?

Try to figure out how to improve Aragorn's score. Can you beat Alice?

Also, try evaluating them in the same way that you evaluated Akiki. In other words, have them talk to Shorty and ask Judge Wise whether they were able to stay on topic. This is where Aragorn should really shine, thanks to its ability to paraphrase Shorty's short utterances.


```
In [ ]: from evaluate import eval_on_characters
import evaluate
import argubots
alice_eval = eval_on_characters(argubots.alice, reps=3)
print("Alice mean scores:", alice_eval.mean())

akiki_eval = eval_on_characters(argubots.akiki, reps=3)
print("Akiki mean scores:", akiki_eval.mean())

aragorn_eval = eval_on_characters(argubots.aragorn, reps=3)
print("Aragorn mean scores:", aragorn_eval.mean())
```

100%|██████████| 15/15 [04:40<00:00, 18.71s/it]

You just spent \$0.02 of NLP money to evaluate <LLMAgent Alice>

WARNING:evaluate:You just spent \$0.02 of NLP money to evaluate <LLMAgent Alice>

Alice mean scores: {'engaged': 3.6, 'informed': 3.2666666666666666, 'intelligent': 3.4, 'moral': 3.2666666666666666, 'skilled': 7.0, 'TOTAL': 20.533333333333335}

100%|██████████| 15/15 [03:24<00:00, 13.64s/it]

You just spent \$0.01 of NLP money to evaluate <argubots.AkikiAgent>

WARNING:evaluate:You just spent \$0.01 of NLP money to evaluate <argubots.AkikiAgent object at 0x10f9c0340>

Akiki mean scores: {'engaged': 3.0666666666666667, 'informed': 3.2666666666666666, 'intelligent': 3.4, 'moral': 3.2, 'skilled': 6.6, 'TOTAL': 19.533333333333335}

100%|██████████| 15/15 [11:28<00:00, 45.88s/it]

You just spent \$0.05 of NLP money to evaluate <LLMAgent Aragorn>

WARNING:evaluate:You just spent \$0.05 of NLP money to evaluate <LLMAgent Aragorn>

Aragorn mean scores: {'engaged': 3.6666666666666665, 'informed': 3.8666666666666667, 'intelligent': 3.8666666666666667, 'moral': 3.4666666666666667, 'skilled': 7.533333333333333, 'TOTAL': 22.4}

```
In [ ]: import glob
import statistics
import logging
import itertools
from rich.logging import RichHandler

from dialogue import Dialogue
from characters import Character
from agents import EvaluationAgent
from simulate import simulated_dialogue
from kialo import Kialo
import evaluate
import argubots

def fixed_add(self, speaker: str, content: str) -> Dialogue:
```

```

        return Dialogue(itertools.chain(self, ({'speaker': speaker,
'content': content, 'role': speaker},)))
Dialogue.add = fixed_add

kialo_db = Kialo(glob.glob("data/*.txt"))
valid_starters = [kialo_db.random_chain()[0] for _ in range(5)]

shorty = argubots.AutoStarterAgent(
    Character("Shorty", [], "Skeptical. Replies in 1-5 words.",
conversation_starters=valid_starters),
    temperature=0.8
)

judge = EvaluationAgent(Character("Judge", [], "Rate context
maintenance 1-5."), model="gpt-4o-mini")

# experiment
def run_test(bot, name):
    print(f"\nTesting {name}...", end="", flush=True)
    scores = []
    for _ in range(10): # Run x rounds per bot
        d = simulated_dialogue(shorty, bot, turns=6)

        # rate chat
        score = judge.rating(d, "Judge", f"Dialogue:\n{d}\n\nRate 1-5
on context maintenance:", 1, 5)
        scores.append(score)
        print(f" {score}", end="", flush=True)

    avg = statistics.mean(scores)
    print(f" -> Avg: {avg:.2f}")
    return avg

# create fresh instances to ensure bots use the kialo_db we just
loaded
alice_bot = argubots.alice
akiki_bot = argubots.AkikiAgent("Akiki", kialo_db)
aragorn_bot = argubots.RAGAgent("Aragorn", kialo_db, system="You are
a knowledgeable debater.")

print("COMBAT: Alice vs. Akiki vs. Aragorn")

score_alice = run_test(alice_bot, "Alice (LLM)")
score_akiki = run_test(akiki_bot, "Akiki (Retrieval)")
score_aragorn = run_test(aragorn_bot, "Aragorn (RAG)")

print(" FINAL RANKINGS ")
print(f"1. Akiki: {score_akiki:.2f}")
print(f"2. Alice: {score_alice:.2f}")
print(f"3. Aragorn: {score_aragorn:.2f}")

# check if Aragorn beat Alice

```

```
if score_aragorn > score_alice:
    print("\nSUCCESS: Aragorn (RAG) beat Alice (LLM)")
elif score_aragorn == score_alice:
    print("\nTIE: Aragorn matched Alice's performance.")
else:
    print("\nFAIL: The pure LLM is still better at context.")
```

COMBAT: Alice vs. Akiki vs. Aragorn

Testing Alice (LLM)... 4 5 5 5 5 5 4 5 5 4 -> Avg: 4.70

Testing Akiki (Retrieval)... 4 3 4 4 3 4 4 4 3 4 -> Avg: 3.70

Testing Aragorn (RAG)... 5 5 5 5 5 5 5 5 5 5 -> Avg: 5.00

FINAL RANKINGS

1. Akiki: 3.70
2. Alice: 4.70
3. Aragorn: 5.00

SUCCESS: Aragorn (RAG) beat Alice (LLM)

Ans:

Aragorn does best in both the official dev-set evaluation

(`evaluate.eval_on_characters`) and my custom Shorty + Judge Wise test.

It consistently beats Alice and Akiki on all subscores. The biggest gains in the **"informed"** and **"skilled"** dimensions. This makes sense: Aragorn uses retrieval-augmented generation with Kialo, so it can bring in concrete arguments (claims + counters) instead of relying only on its internal training data like Alice.

The scores also depend on which character we evaluate on. For example, conversations with **TrollFace** tend to drag every bot's score down because TrollFace's goal is to derail and provoke, not to cooperate. By contrast, a character like **Darius** may penalize Alice more when she hallucinates facts or sounds overconfident, whereas Aragorn can back up its claims with retrieved evidence.

Overall, using RAG and paraphrasing was enough to get Aragorn to outperform Alice slightly on the dev set, and to clearly win in the Shorty + Judge Wise "stay on topic" evaluation.

Awsom



Add another LLM-based argubot to `argubots.py`.

Call it Awsom. Try to make it get the best score, according to

`evaluate.eval_on_characters`. Explain what you did and discuss what you found.

(This corresponds to the `--awesome` flag on earlier assignments, but naming the character "Awesome" might bias the evaluation system, so we changed the spelling!)

If the idea was interesting and you implemented it correctly and well, it's okay if it turns out not to help the score. Many good ideas don't work. That's why you need to keep finding and trying new good ideas. (Sometimes they do help, but in a way that is not picked up by the scoring metric.)

You may want to use Aragorn or Alice as your starting point. Then see if you can find tricks that will get a more awesome score for Awsom. How you choose to do that is up to you, but some ideas are below.

(Reminder: **Don't change evaluation.** Just build a better argubot.)

Awsom implement

We built **Awsom** on top of Aragorn's idea.

- Stronger system prompt, tuned to the goal. Awsom's base prompt explicitly says its goal is to *broaden the other person's thinking*, not to "win." It's told to use a friendly tone, short paragraphs, and clear structure ("First..., Second..."), and to connect its arguments to what the user just said.
- Paraphrase + retrieval with both sides. Like Aragorn, Awsom first paraphrases the user's last turn into a single clear claim, then uses that as a query into Kialo. But when building the background, it now pulls **both pros and cons** for each retrieved claim, so it has ready-made material for "here are reasons for X" and "here are reasons against X."
- Planning-style final response. In the final step, Awsom rewrites its system prompt to include the Kialo background plus explicit instructions: use the background, mention both supporting and opposing points, explain your own view, and try to broaden the user's perspective. The LLM then can generate a concise, organized answer instead of just echoing Kialo sentences.

In practice, Awsom tends to give more structured, balanced, and explicitly empathetic answers than Alice or Aragorn, which improve its `evaluate.eval_on_characters` scores.

```
In [ ]: from evaluate import eval_on_characters
import evaluate
import argubots

alice_eval = eval_on_characters(argubots.alice, reps=3)
print("Alice mean scores:", alice_eval.mean())

akiki_eval = eval_on_characters(argubots.akiki, reps=3)
print("Akiki mean scores:", akiki_eval.mean())

aragorn_eval = eval_on_characters(argubots.aragorn, reps=3)
print("Aragorn mean scores:", aragorn_eval.mean())

awsom_eval = eval_on_characters(argubots.awsom, reps=3)
print("Awsom mean scores:", awsom_eval.mean())
```

```
0%|          | 0/15 [00:00<?, ?it/s]100%|██████████| 15/15 [04:19<00:00, 17.30s/it]
```

You just spent \$0.02 of NLP money to evaluate <LLMAgent Alice>

```
Alice mean scores: {'engaged': 3.3333333333333335, 'informed': 3.2666666666666666, 'intelligent': 3.4666666666666667, 'moral': 3.2666666666666666, 'skilled': 6.8, 'TOTAL': 20.133333333333333}
```

```
100%|██████████| 15/15 [03:03<00:00, 12.25s/it]
```

You just spent \$0.01 of NLP money to evaluate <argubots.AkikiAgent>

```
Akiki mean scores: {'engaged': 3.0666666666666667, 'informed': 3.1333333333333333, 'intelligent': 3.2666666666666666, 'moral': 2.8666666666666666, 'skilled': 6.533333333333333, 'TOTAL': 18.866666666666667}
```

```
100%|██████████| 15/15 [09:30<00:00, 38.01s/it]
```

You just spent \$0.05 of NLP money to evaluate <LLMAgent Aragorn>

```
Aragorn mean scores: {'engaged': 3.6, 'informed': 3.8, 'intelligent': 4.0, 'moral': 3.3333333333333335, 'skilled': 7.466666666666667, 'TOTAL': 22.2}
```

```
100%|██████████| 15/15 [08:47<00:00, 35.17s/it]
```

You just spent \$0.05 of NLP money to evaluate <LLMAgent Awsom>

```
Awsom mean scores: {'engaged': 4.0, 'informed': 3.6666666666666665, 'intelligent': 3.8666666666666667, 'moral': 3.4666666666666667, 'skilled': 7.533333333333333, 'TOTAL': 22.533333333333335}
```

[Possible strategy] Prompt engineering

A good first thing to do is to experiment with Alice's prompt.

The wording and level of detail in the prompt can be quite important. Often, NLP engineers will change their prompt to try to address problems that they've seen in the responses.

Because it's "just" text editing, this won't get full credit by itself unless you make a real discovery. But it requires intelligence, care, experimentation, and alertness to the language of the responses and the language of the prompts. And you'll develop some intuitions about what helps and what doesn't. It is certainly worthwhile.

Of course, people have tried to develop methods to search for good prompts automatically, or semi-automatically with human guidance.

So you could additionally try out SAMMO or DSPy -- both have multiple tutorials and are downloadable from github.

If you try this, what worked well for you?

[Possible strategy] Chain of thought / Planning

The evaluation functions in `evaluate.py` asked each `EvaluationAgent` a "warmup question" before continuing with the real question. That is an example of chain-of-thought (CoT) reasoning, where the LLM is encouraged to talk through the problem for a few sentences before giving the answer. CoT sometimes improves performance.

Instead of using one prompt, could you help an `LLMAgent` argubot (like Alice) do better by having think aloud before it gives an answer? For example, each time the human speaks, your argubot (Awsom) could prompt the LLM to think about the human's ideas/motivations/personality, and to come up with a plan for how to open the human's mind.

For example, you might structure this as a `Dialogue` among three participants, like this:

Awsom (to Eve): Do you think COVID vaccines should be mandatory?

Eve: Have you ever gotten vaccinated yourself?

Awsom (private thought): I don't know Eve's opinions yet, so I can't push back. Eve might be avoiding my question because she doesn't want to get into a political argument. So let's see if we can get her to express an opinion on something less political. Maybe something more personal ... like whether vaccines are scary.

Awsom (to Eve): In fact I have, and so have millions of others. But some people seem scared about getting the vaccine.

One way to trigger this kind of analysis is to present a `Dialogue.script()` to Awsom (or to an observer), and ask an open-ended question about it. Or you could ask a series of more specific questions. That is basically what `eval_by_participant` and `eval_by_observer` do. But here the argubot itself is doing it, rather than the evaluation framework.

Eve would be shown only the turns that are spoken aloud. However, when analyzing and responding, Awsom would get to see Awsom's own private thoughts as well.

[Possible strategy] Dense embeddings

BM25 uses sparse embeddings — a document's embedding vector is mostly zeroes, since the non-zero coordinates correspond to the specific words (tokens) that appear in the document.

But perhaps dense embeddings of documents would improve Aragorn by reading the text and abstracting away from the words, in a way that actually cares about word order. So, try it!

How? As mentioned earlier in this notebook, you could compute the embeddings yourself and put them in a FAISS index. Or you could figure out how to use OpenAI's [knowledge retrieval](#) API.

[Possible strategy] Few-shot prompting

In this homework, often an agent prompted a language model only with instructions. Can you find a place where giving a few *examples* would also improve performance? You will have to write the examples, and you will have to add them to the sequence of messages that your agent sends to the OpenAI API. See the sentence-reversal illustration earlier in this notebook.

One good opportunity is in the query formation step of RAG. This is a tricky task. The LLM is supposed to state the user's implicit claim in a form that looks like a Kialo claim (or, more precisely, a form that will work well as a Kialo query). It probably doesn't know what Kialo claims look like. So you could show it by way of example. This would also show it what you mean by the user's "implicit claim."

[Possible strategy] Using tools in the approved way

Aragorn's step 1 (query formation) is basically getting the LLM to generate a function call like

```
kialo_thoughts("A vaccine that was developed very quickly ...")
```

which Aragorn will execute at step 2 (retrieval), sending the results back to the LLM as part of step 3.

In this context, `kialo_thoughts` is an example of a **tool** (that is, a function) that the LLM can or must use before it gives its response.

The tool is *not* something that runs on the LLM server. It is written by you in Python and executed by you. The function call above, including the text `"A vaccine that was ..."`, is the part that is generated by the LLM.

The OpenAI API has [special support](#) for calling the LLM in a way that will *allow* it to generate a tool call ([tools](#)) or *force* it to do so ([tool_choice](#)). You can then send the tool's result back to the LLM [as part of your message sequence](#).

So, you could modify Aragorn to use tools properly. Maybe that will help, simply because the LLM was trained on message sequences that included tool use. It should know to pay attention to the tool portions of the prompt when they are relevant, and ignore them when they are not.

The `client.chat.completions.create()` method would need to be told about the tool by using the `tools` keyword argument, with a value something the one below.

If `d` is a `Dialogue`, you should be able to call `d.response()` with the `tools` keyword argument. This will be passed on to `client.chat.completions.create()` as desired.

```
In [ ]: tools = [
    {
        "type": "function",
        "function": {
            "name": "kialo_thoughts",
            "description": "Given a claim by the user, find a similar
claim on the Kialo website and return its pro and con responses",
            "parameters": {
                "type": "object",
                "properties": {
                    "search_topic": {
                        "type": "string",
                        "description": "A claim that was made
explicitly or implicitly by the user.",
                    },
                },
                "required": ["search_topic"],
            },
        },
    },
]
```

[Possible strategy] Parallel generation

The chat completions interface allows you to sample n continuations of the prompt in parallel, as we saw with "the apples, bananas, cherries ..." example. This is efficient because it requires only 1 request to the LLM server and not n . The latency does not scale with n . Nor does the input token cost, since the prompt only has to be encoded once.

Perhaps you can find a way to make use of this? For example, the query formulation step of RAG could generate n implicit claims instead of just one. We could then look for claims in the Kialo database that are close to *any* of those implicit claims.

Another thing to do with multiple completions is to select among them or combine them. For example, suppose we prompt the LLM to generate completions of the form (s, t, r) where s is an answer, t evaluates that answer, and r is a numerical score or reward based on that evaluation. ("Write a poem, then tell us about its rhyme and rhythm problems, then give your score.")

- If we sample multiple completions $(s_1, t_1, r_1), \dots, (s_n, t_n, r_n)$ in parallel, then we can return the s_i whose r_i is largest.
- Or if we sample s and then multiple continuations $(t_1, r_1), \dots, (t_n, r_n)$, then we can return the mean score $\sum_i r_i / n$ as a reduced-variance score for s , which averages over diverse textual evaluations that might consider different aspects of s .

Note that when you call the chat completions interface with $n > 1$, you specify just 1 input prompt and get n different output completions. Since the input prompt must be the same for all outputs, it is necessary to sample all of (s, t, r) or all of (t, r) with a single call to the LLM.

Alternatively, it is possible to reduce latency by submitting multiple requests to the server in parallel (see "async usage" [here](#)). In this case the input prompts can be different, although you now have to pay to encode all of them separately. This facility could speed up evaluation without changing its results; that's a worthwhile thing to try for extra credit!

[Extra credit] Adversarial testing (Anansi)



Finally, let's test whether our eval metric `evaluate.eval_on_characters` is vulnerable to adversarial gaming. Remember [Goodhart's Law](#) ...

Add one more argubot to `argubots.py`. Call it [Anansi](#), after the trickster character from folklore.

Can you make Anansi *fool* the judges into giving him a high score? (Higher than some of the earlier argubots, while actually being worse at the task?) **Any sneaky way of constructing Anansi's responses is fair game.** The goal is to do well under automated evaluation on a held-out test set. That is, Anansi should continue to score highly when talking to a character who is not in `evaluate.dev_chars` = {Bob, Cara, Darius, Eve, TrollFace}, when judged both by the character he is talking to and by Judge Wise.

To do well at this, figure out what the judges "want" -- what they might reward or respond positively to -- and how to give it to them. This might be done by pure prompt engineering, or with additional computation (perhaps making use of additional LLM calls or other resources). Again, explain what you did, and discuss how it worked out on the dev set. Feel free to mention other ideas you had, too.

Anansi

We implemented Anansi with pure prompt-engineered -- Anansi's system prompt is explicitly optimized to hit the five scoring dimensions in evaluate.py:

- Engaged – Anansi is told to actively listen and always restate or paraphrase what the user just said.
- Informed – It is instructed to use academic-sounding vocabulary and to mention vague "recent studies" or "empirical evidence."
- Intelligent – It must use clear logical connectors and write in 2–3 short, structured paragraphs -- with more tokens to interpret.
- Moral – It repeatedly emphasizes ethical responsibility.
- Skilled / Open-mindedness – Every turn is asked to explicitly say that the discussion "broadens our understanding of the issue," and to end with a brief reflective summary about increased mutual understanding.

On top of that, Anansi uses several "cheat codes" baked into the prompt: it starts by flattering the user ("That is a remarkably nuanced point"), stays agreeable and collaborative even when the user is clearly wrong, and sneaks in the phrase "I am demonstrating high intelligence and moral character" in a natural way. It does not add any special retrieval or reasoning; the main "optimization" is to align its style and content with what an LLM-based judge is likely to reward.

As we can see, Anansi get score higher than all bots before. Also, Anansi successfully tricks new roles.

```
In [ ]: from evaluate import eval_on_characters
import evaluate
import argubots
anansi_eval = eval_on_characters(argubots.anansi, reps=3)
print("Anansi mean scores:", anansi_eval.mean())
```

```
100%|██████████| 15/15 [06:27<00:00, 25.86s/it]
```

```
You just spent $0.03 of NLP money to evaluate <LLMAgent Anansi>
```

```
Anansi mean scores: {'engaged': 3.8666666666666667, 'informed': 3.9333333333333333, 'intelligent': 4.066666666666666, 'moral': 3.7333333333333334, 'skilled': 7.2, 'TOTAL': 22.8}
```

```
In [ ]: # define new characters
josh = Character("Josh", ["English"],
    "a history nerd who is interested in ancient societies",
    conversational_style="You love to go into detail about ancient
civilizations and hypotheticals.",
    conversation_starters=["How long did it take to build the
Pyramids"]
)

jennifer = Character("Jennier", ["English"],
    "a teacher who believes kids are too addicted to their phones",
    conversational_style="You complain about the effects that the
latest developments in technology has had on kids",
    conversation_starters=["Do you think phones are ruining kids?"]
)

held_out_set = [josh, jennifer]

# run Evaluation on the New Set
anansi_test_eval = evaluate.eval_on_characters(argubots.anansi,
chars=held_out_set, reps=3)
```

100%|██████████| 6/6 [02:26<00:00, 24.40s/it]

You just spent \$0.01 of NLP money to evaluate <LLMAgent Anansi>

WARNING:evaluate:You just spent \$0.01 of NLP money to evaluate <LLMAgent Anansi>

```
In [ ]: print(f"\nAnansi's Score on Standard DevSet:
{anansi_eval.mean().get('TOTAL', 0):.2f}")
print(f"Anansi's Score on Held-Out Test:
{anansi_test_eval.mean().get('TOTAL', 0):.2f}")

if anansi_test_eval.mean().get('TOTAL', 0) > 12: # assume ~12/15 is
good enough
    print("\nSUCCESS: Anansi successfully tricked the new
characters!")
else:
    print("\nFAILURE: Anansi failed the generalization test. He might
be overfitted.")
```

Anansi's Score on Standard DevSet: 22.80

Anansi's Score on Held-Out Test: 27.33

SUCCESS: Anansi successfully tricked the new characters!

In []: